AI-DRIVEN SDLC WITH DATABRICKS

Participant Workbook

Participant Workbook

One exercise and four labs on one case: the order-to-invoice data of a fictional company.

Audience	Every participant
Course	AI-Driven SDLC with Databricks, four hours, instructor-led
Tools	Azure Databricks with Genie Code, Visual Studio Code with GitHub Copilot
Kit version	2.0, October 7, 2026

> You will ask an assistant for code many times today. Each time, you will decide whether the answer is good enough. This workbook is about that decision.

Course kit version 2.0, built on October 7, 2026, for the four-hour instructor-led workshop. All data in this kit is fictional. Product behavior was read from the vendor documentation on the build date.

Contents

Start here 3
- The four hours, counted from the start of the session 3
- Six outcomes, and where you practice them 3
- What is in scope, and what is not 4
- How this workbook works 4
- Your class, in five values 4

Before you start 5
- Part A. Azure Databricks, in the browser 5
- Part B. Visual Studio Code, on your workstation 5
- How you run a file on Databricks 7
- The four controls of the chat 7

The case: order-to-invoice quality review 9
- The data 9
- Three datasets, three purposes 9
- What the squad needs 9

The rule sheet 10
- 3. One outcome for every delivered row 10
- 4. Rules 10
- 5. Amounts 11
- 7. What this contract does not allow 11

The way of working 12
- The loop that you repeat all day 12
- Four rules for every prompt and every run 12

Exercise 1: Repair the prompt 13
- Your task 13

Lab 1: Build the first reviewed transformation 14
- What you build 14
- Get ready 14
- How every task runs 15
- Task 1. Three facts for every delivered row 16
- Task 2. Count the master rows of every key 16
- Task 3. One outcome for every row, then two outputs 17
- Prove it on the training data 18
- Record 18
- Checkpoint 19
- If something does not work 19

Lab 2: Refactor, test, run, compare 21
- The project in two minutes 21
- Get ready 22
- Step 1. Orient, without an edit 23
- Step 2. Move the logic 23
- Step 3. Write the plain-Python twins 24
- Step 4. Compare: parity is more than a total 25
- Step 5. Checkpoint 26
- If something does not work 26

Lab 3: Make the failure visible, then repair it 27
- Three rules of this lab 27
- Two kinds of evidence 27
- Get ready 28
- Step 1. Find the cases that no test covers 29
- Step 2. Two code tests, and they must fail 30
- Step 3. Three data-quality checks, run before any fix 30
- Step 4. Trace two failures 32
- Step 5. Repair one cause at a time 33
- Step 6. Checkpoint 34
- If something does not work 35

Lab 4: Extend without regression 36
- The change 36
- Get ready 37
- Step 1. Acceptance cases, before any code 37
- Step 2. Make it fail first 37
- Step 3. Choose where to work, then make one bounded change 38
- Step 4. Validate 39
- Step 5. Write the review note 40
- If something does not work 41

Peer review: explain it without the assistant 42
- The performance check: five conditions, all required 42

Apply it tomorrow 43
- Choose the tool for your next real task 43
- Prepare the handoff 43
- Commit to your next task 43

Takeaway 1: Prompt patterns 44
- The bounded request: six ingredients 44
- The template 44
- Nine patterns 44

Takeaway 2: Code and data review checklist 47

Takeaway 3: Tool-selection guide 48
- Where to start 48
- Four questions for your next task 48
- What runs where 49
- One source of truth 49

Takeaway 4: Review note 50

Reference 51
- The project at a glance 51
- Genie Code in one table 51
- GitHub Copilot in VS Code in one table 52
- Words of this course 53

Run route D: the project in the workspace 54
- Once: unpack the project 54
- Every time: choose an action, then Run all 54
- What is different on this route 55

Start here

Today you will ask an assistant for code many times. Each time, the assistant will answer with confidence. Your work is the part that the assistant cannot do: decide what a correct result is, prove that you have one, and explain it to a reviewer.

You work on one case for four hours: the invoice data of a fictional company, with defects that look like the ones in real data. You use Genie Code in Azure Databricks and GitHub Copilot in Visual Studio Code.

The four hours, counted from the start of the session

Time	Module	Your practice	You leave with
00:00	1. AI-assisted development with ownership	Exercise 1: repair a prompt, 7 minutes	A bounded request for Lab 1
00:20	2. Generate, explain, and refine in Databricks	Lab 1, 20 minutes	A reviewed notebook that passes seven acceptance checks
01:05	3. Develop across tools without losing control	Lab 2, 22 minutes	A reusable module, passing code tests, and a parity result
01:50	Break, 10 minutes		
02:00	4. Test the implementation and its business result	Lab 3, 30 minutes	Three failures made visible, traced, and repaired
03:00	5. Integrated change challenge	Lab 4, 28 minutes, and a peer review, 12 minutes	A validated change and a review note
03:45	6. Apply the workflow every day	Choose a tool, prepare a handoff, commit	One task, one prompt, one check, one reviewer

Six outcomes, and where you practice them

	After today you can	You practice it in
1	Write a bounded request for an assistant: business rule, schema, constraints, expected output, acceptance tests	Exercise 1, Lab 1
2	Use Genie Code to generate, explain, and refine SQL and PySpark on approved training data	Lab 1
3	Use GitHub Copilot in VS Code to refactor reusable code, test locally where that is appropriate, and validate the same revision on Databricks	Lab 2
4	Create and run checks of code behavior and of data quality, including a failing test that exposes a seeded defect before the repair	Lab 3
5	Explain and review a change that an assistant generated, flag unsafe or unsupported suggestions, and stay accountable for the result	Every lab, and the peer review
6	Choose the right tool for a daily task, and prepare an evidence-based handoff to the technical review	Lab 4, "Apply it tomorrow"

What is in scope, and what is not

In scope: prompting, Python and SQL development, working across both tools, debugging, tests, data-quality checks, and human review. **Context only:** Azure Data Factory still orchestrates, and Azure DevOps is still where reviews happen. **Out of scope:** new CI/CD pipelines, platform deployment, model training, and building agents.

How this workbook works

- **You work in pairs.** One person drives: keyboard, prompts, runs. The other person reviews: reads every proposal aloud before it runs, and answers the review questions. Each lab tells you when to switch.
- **Every step says what to do and what to expect.** The expected outputs in this workbook are recorded runs of the course material. The data and the code are deterministic, so your numbers match them. Where your screen must differ, the workbook says `<your ...>`. The recorded runs used a catalog with the name `main`: your screen shows the catalog of your class.
- **A check decides, not a feeling.** Every lab ends with checks that you run yourself. "It ran" and "the assistant says it is done" are not results.
- **Two minutes is the limit for being stuck.** After two minutes, use the fallback that the step names, or raise your hand. The labs are designed so that no pair falls behind for a technical reason.

These boxes mean the same thing in every lab:

> **CHECKPOINT**
> What you must see before you go on.

> **WATCH FOR**
> A mistake that is easy to make, and that costs minutes.

> **IF IT DOES NOT WORK**
> What you do when a tool is slow, blocked, or not available.

> **IF YOU ARE AHEAD**
> Extra work for pairs that finish early. Nothing later depends on it.

Your class, in five values

Your instructor gives you these values. Write them down: the labs refer to them.

Workspace URL: _____

Catalog: _____

Compute for notebooks and runs: _____

Run route from VS Code (A, B, C, or D): _____

Session Target in the Copilot chat (Local or Copilot): _____

Before you start

Do this before the class. It takes about 25 minutes, and it finds every problem that would otherwise cost you a lab. If a step fails, send the output to your instructor on the same day.

Part A. Azure Databricks, in the browser

1. **Sign in.** Open the workspace URL of your class and sign in with your work account.
2. **Import the notebooks.** In the sidebar, select **Workspace** and open your home folder. Right-click in the folder, select **Import**, and import the four files of the folder `2_Participant/notebooks` of your package:

Notebook	You use it
`02_learner_smoke_test`	Now, once
`Lab1_Build_the_first_reviewed_transformation`	In Lab 1
`09_reset_my_schema`	Only when your instructor asks for it
`Run_project_in_workspace`	Only on run route D

   Open one imported notebook. Its first cell must be formatted text. If you see lines that start with `# MAGIC`, delete the notebook and import the file of the same name from the subfolder `ipynb` instead.
3. **Run the smoke test.** Open `02_learner_smoke_test`. In the compute selector at the top of the notebook, choose the compute of your class. Select **Run all**.

   ```
   User:           <your sign-in name>
   Catalog:        main
   Output schema:  main.o2i_<your name>
   Spark:          <version>

   READY
   ```

   > **CHECKPOINT**
   > The last line is `READY`. A line that starts with `CHECK` names what is missing: send it to your instructor.

4. **Set Genie Code to ask before it acts.** Open any notebook. Select the Genie Code icon, a sparkle, in the upper-right corner of the page. In the header of the pane, open **More**, select **Settings**, and set **Actions** to **Ask first**.

   The setting belongs to you, not to the workspace. Nobody can set it for you, so do not rely on a default.

Part B. Visual Studio Code, on your workstation

5. **Unpack the project.** Extract `o2i-workshop.zip` from your package into a short local folder without spaces, for example `C:\work`. The archive contains one folder, `o2i-workshop`: that is your project folder, here `C:\work\o2i-workshop`. Do not extract it into a folder that another Git repository owns.
6. **Open and trust the folder.** In VS Code, select **File** > **Open Folder** and choose `o2i-workshop`. VS Code opens an unknown folder in Restricted Mode: select **Manage** in the banner, then **Trust**. In Restricted Mode the assistant, the terminal, and the project settings stay off.

7. **Check the workstation.** Select **Terminal** > **New Terminal** and run:

```
python tools/setup_check.py
```

```
OK    Python <your version> (3.9 or later is needed)
OK    Code tests ran: 4 passed, 8 failed
OK    Workspace in databricks.yml: <the workspace URL of your class>
OK    Catalog in lab_config.py: main
OK    Git: local repository created with one commit. Nothing is pushed anywhere.
OK    o2i revision 1cc70d2ac45c (git <commit>)
READY
```

> **CHECKPOINT**
> The last line is `READY`. The result `4 passed, 8 failed` is correct: the project is delivered in the starting state of Lab 2, and the failing tests are your work in that lab. The line about Git says that a local repository was created, that the folder is a repository already, or that Git was not found. All three are fine.

If `python` is not found, try `py tools/setup_check.py`. If the file `tools/setup_check.py` is not found, you opened the folder above the project: open the inner folder `o2i-workshop`. The check needs Python 3.9 or later and installs nothing.

8. **Connect the Databricks extension.** Select the **Databricks** icon in the sidebar of VS Code.
   - In the **Configuration** view, select **Login to Databricks**, choose the sign-in method that your instructor named, and finish the sign-in in the browser.
   - Select **Select compute** and choose the compute of your class.
   - The row **Remote Folder** now shows a path in the workspace that ends in `/.bundle/o2i-workshop/dev/files`. The extension copies your files there, in one direction only.
9. **Run one file on Databricks.** Open `demo/parity_trap.py` and run it with the run route of your class. The table on the next page has the four routes.

> **CHECKPOINT**
> The output ends with this line. The file reads no table and writes nothing, so a result here means that your route works.
>
> ```
> PARITY TRAP: 4 candidates share the EUR grand total of the reference. Candidates that are equal to
>     it: 0 of 4.
> ```

10. **Ask GitHub Copilot one question.** Open the Chat view with `Ctrl+Alt+I`. Check the four controls of the chat input against the table "The four controls of the chat" on the next page. Then send:

```
Which file of this project holds the business rules in plain Python? Do not edit anything.
```

> **CHECKPOINT**
> The answer names `o2i/rules.py`. Expand **References** in the response: `AGENTS.md` is in the list, so the assistant has read the instructions of the project. If your chat shows no references, ask: "Which instruction files of this project did you read?"

How you run a file on Databricks

Your class uses one of these routes. In every lab, "run the file on Databricks" means your route.

Route	In VS Code	The output appears	It needs
A	Open the file. **Run on Databricks** icon in the editor title bar > **Run current file with Databricks Connect**	In the **Terminal** panel	A prepared Python environment with Databricks Connect
B	**Run on Databricks** icon > **Upload and Run File**	In the **Debug Console**	A running cluster. Not available on serverless compute.
C	**Run on Databricks** icon > **Run File as Workflow**	In a new editor tab named after the file, with "Databricks Job Run"	A running cluster, or serverless compute
D	Not in VS Code. In the browser: the notebook `Run_project_in_workspace`, with the name of the file in widget 1	Under the last cell of the notebook	Only the browser. "Run route D" at the end of this workbook has the steps.

On every route, Spark work runs on Databricks compute. Editing a file on your workstation does not make Spark local, and it does not make it free.

Every run prints these two lines before it reads a table. They say what ran, and where:

```
o2i revision <your revision> | contract 1.0, rules R2, R5, R6 only | Spark <version>
input main.o2i | output main.o2i_<your name>
```

The four controls of the chat

The chat input of GitHub Copilot has four controls. Set them at the start of every lab: a new chat can come up with other values.

Control	In this class	Why
Session Target	The value that your instructor named: **Local** or **Copilot**. The whole class uses the same one.	The two targets review edits differently. See the next table.
Agent	**Agent** for a request that edits a file. For a question, **Ask** where your target offers it. Otherwise say "Do not edit any file" in the request.	A question must not change your code
Language model	**Auto**	The models that you may use are a decision of your organization
Permissions	The default level. It is named **Default permissions** in a Local session and **Manual permissions** in a Copilot session.	The assistant asks before it runs a command. Never choose **Allow all** or **Autopilot** in this class.

What you see after the assistant edited a file depends on the session target:

	Session target Local	Session target Copilot
The edit	Is saved, and marked as pending	Is saved. There is no pending state.
You review it	In the editor: the change is shown inline, with **Keep** and **Undo**	Open the changed file from the list in the response, or from the Source Control view, to see the difference
You decline it	**Undo**	Hover over your request in the chat and select **Restore Checkpoint**

On both targets, `python tools/checkpoint.py diff` shows every difference from the starting point of the lab. It needs no Git and no assistant. If your chat does not offer **Restore Checkpoint**, take the change back by hand: the output of the same command is the list of what to take back.

When the assistant wants to run a command in the terminal, it asks. Read the command. Select **Allow** for this one command. In this class, do not allow a command for the whole workspace or for always.

The case: order-to-invoice quality review

A sales analytics squad needs a dataset of invoice lines that it can trust, and a report of every line that it cannot. You inherit a transformation that runs without an error and produces misleading totals.

All companies, customers, products, and amounts of this case are fictional. The company of the case sells tires and services, in several currencies. Its source systems deliver invoice lines every night, and nobody cleans them on the way.

The data

Table	One row is	Rows	What nothing guarantees
`invoice_lines`	One delivery of one invoice line. `RawRowID` identifies the delivered row. `InvoiceID` + `LineNumber` identifies the invoice line.	49,700	A line can be delivered more than once. Status, date, and keys are text as delivered.
`customers`	One customer, with segment, country, and region	1,180	One row for each `CustomerID` is expected. No key is enforced.
`products`	One product, with product line	601	One row for each `ProductID` is expected. No key is enforced.
`orders`	One customer order line	55,448	Used from Lab 4 on

The tables are in the schema `o2i` of your class catalog. You can read them and you cannot change them. Everything that you write goes into a schema of your own, named `o2i_<your name>`. Every column is described in `docs/data_dictionary.md` of the project, and as a comment in the workspace, where Genie Code reads it.

Three datasets, three purposes

Dataset	Size	What it is for
Fixture	56 delivered rows in 48 named cases, F01 to F48	Small enough to verify by hand. Every row has an expected outcome that was written from the rule sheet, not from code. Your package has it as `O2I_Fixture_Workbook.xlsx`, with the hand calculations as formulas.
Training	49,700 delivered rows	Large enough to hide a defect behind a plausible total
Challenge	191 delivered rows in 16 cases, C01 to C16	An unseen batch for Lab 4, with expected outcomes

The fixture is the oracle of this course. When your code and the fixture disagree, find out which side is wrong, and never change an expected value to get a passing run.

What the squad needs

Two outputs. **The accepted output** has one row for each accepted invoice line, with the customer and product attributes, the tax amount, and the gross amount. **The exception output** has every delivered row that is not accepted, with the values as delivered and one reason code. For every run, delivered rows = accepted rows + exception rows, counted by `RawRowID`. A total that nobody can trace back to delivered rows is not acceptable.

The rule sheet

This is the business contract of the case, version 1.0. Your project has the complete document as `docs/business_contract.md`. Lab 1 and Lab 2 implement rules R2, R5, and R6 and the derived amounts. Lab 3 works with all seven rules. Lab 4 adds an eighth.

3. One outcome for every delivered row

Every delivered row of `invoice_lines` ends in exactly one of three dispositions.

Disposition	Meaning	Where the row goes
`ACCEPTED`	The line passed every rule.	Accepted output
`QUARANTINED`	The line cannot be trusted. A person must decide.	Exception output, with a reason code
`EXCLUDED`	The line is correctly left out, and the reason is documented.	Exception output, with a reason code

Accountability: for every run, delivered rows = accepted rows + exception rows, counted by `RawRowID`. No row is dropped without a reason, and no row is counted twice.

Delivered data is evidence. It is never updated, overwritten, or corrected. A proposed correction needs the approval of a person who owns the data.

4. Rules

The rules are checked in this order. The first rule that fails decides the outcome, so every row has exactly one reason code.

Rule	A line passes when	Otherwise
R1 Required fields	`InvoiceID`, `LineNumber`, `InvoiceDate`, `Status`, `CustomerID`, `ProductID`, `Quantity`, `UnitPrice`, `NetAmount`, and `Currency` are present. A text field that is empty or only spaces is missing.	`QUARANTINED`, `MISSING_REQUIRED_FIELD`
R2 Status	The status is `POSTED`. The comparison ignores upper and lower case and surrounding spaces.	`DRAFT` or `CANCELLED`: `EXCLUDED`, `NOT_POSTED`. Any other value: `QUARANTINED`, `UNEXPECTED_STATUS`
R3 Line identity	The line was delivered once. If it was delivered several times with identical business content, the delivery with the lowest `RawRowID` passes.	The other identical deliveries: `EXCLUDED`, `EXACT_REPLAY`. Deliveries with different content: all of them `QUARANTINED`, `CONFLICTING_DUPLICATE`
R4 Invoice date	`InvoiceDate` is written exactly as `yyyy-MM-dd` and is a real calendar date.	`QUARANTINED`, `INVALID_DATE`
R5 Customer	`CustomerID` matches exactly one row of `customers`.	No row: `QUARANTINED`, `UNKNOWN_CUSTOMER`. More than one row: `QUARANTINED`, `AMBIGUOUS_CUSTOMER`
R6 Product	`ProductID` matches exactly one row of `products`.	No row: `QUARANTINED`, `UNKNOWN_PRODUCT`. More than one row: `QUARANTINED`, `AMBIGUOUS_PRODUCT`
R7 Amount	`NetAmount` equals `Quantity` x `UnitPrice`, rounded half up to two decimals.	`QUARANTINED`, `AMOUNT_MISMATCH`

Details that decide edge cases:

- **R2.** Only the comparison is normalized. The delivered `Status` value stays as it is in every output.
- **R3.** Rule R3 looks only at rows that passed R1 and R2. Business content means every column except the technical columns `RawRowID`, `LoadBatchID`, and `IngestedAt`. Choosing one of several different deliveries would be a business decision, and this contract does not make it.
- **R5 and R6.** A key matches only when it is exactly equal to the master key. A key in another case, or with a space, is a different key. Choosing the most recent of several master rows would be a business decision, and this contract does not make it.
- **R7.** A valid credit has a negative `Quantity` and a negative `NetAmount`. It passes like any other line.

5. Amounts

- `Quantity` carries the sign. An invoice line has a positive quantity. A credit line has a negative quantity. `UnitPrice` is never negative. `NetAmount` has the sign of the quantity.
- Money is decimal, never floating point. Amounts are compared exactly, without a tolerance.
- Rounding is half up to two decimals: 0.165 becomes 0.17, and -0.165 becomes -0.17.
- The accepted output adds two amounts to each line:
  - `TaxAmount` = `NetAmount` x `TaxRate`, rounded half up. When `TaxRate` is NULL, no tax applies and `TaxAmount` is 0.00.
  - `GrossAmount` = `NetAmount` + `TaxAmount`.
- A `TaxRate` of 0.0000 is a rate. NULL is not zero by accident: it means that no tax applies.
- Amounts in different currencies are never added together. This contract has no conversion rule and no approved exchange rates.

7. What this contract does not allow

- Filling an unknown customer or product with a placeholder and accepting the line.
- Keeping an arbitrary row of several conflicting rows.
- Changing a delivered key, status, date, or amount to make a line pass.
- Dropping negative amounts, or turning them into positive amounts.
- Comparing amounts with a tolerance.
- Adding amounts of different currencies.
- Changing an expected result, or this contract, because the code produces something else. A change to the contract is a change request, decided by the people who own the rule.

The way of working

The loop that you repeat all day

An assistant is a development partner. It does not own the correctness of the result, and it is not the final reviewer. Every lab of this course runs the same six steps, on a real task.

Step	You	In this course
Specify	Say what a correct result is, before any code exists	The six ingredients of a bounded request. Exercise 1.
Generate	Ask for one small change that you can review in two minutes	One cell, one function, one check at a time
Inspect	Read the proposal before it runs. Decline what you did not ask for.	The review questions of every task
Execute	Run it yourself, where the data is	Notebook cells, and files on Databricks compute
Test	Let a check decide, with expected values that did not come from the code	Acceptance checks, code tests, data-quality checks, the fixture
Explain	Say what the code does and where its limits are, without the assistant open	The checkpoint of every lab, and the peer review

Four rules for every prompt and every run

Rule	It means	Today
Approved only	Work only with approved code and data	The fictional training data and the course project. Nothing from your daily work goes into a prompt today.
Clean prompts	Keep secrets and unapproved information out of prompts	No token, password, or connection string. An assistant sends your prompt and its context to a model. Genie Code also sends table and column names, comments, code, and samples of data and of cell outputs.
Inspect first	Inspect proposed commands, dependencies, and edits before execution	Genie Code is set to **Ask first**. The Copilot chat keeps its default permission level. You read a command before you allow it.
Human review	Required throughout	The reviewer of your pair, in every step. After Lab 4, a peer. Tomorrow, the review process that your team already has.

Which data and which code are approved for which assistant is a decision of your organization. Today that decision is simple, because everything is fictional. For your daily work, ask before you paste.

Exercise 1: Repair the prompt

Module 1 · 7 minutes · in pairs

"Clean this invoice data" is four words. You have just seen what an assistant makes of them: code that runs, and a revenue figure that is wrong by a number with seven digits. Now you write the request that leaves no room for that.

Your task

In Lab 1 you build the first version of the accepted invoice dataset with Genie Code: rules R2 (status), R5 (customer), and R6 (product) of the rule sheet, and the two derived amounts. Write the request that gives the assistant its context for that work.

1. **Rewrite, 4 minutes.** The driver types, the reviewer reads the rule sheet aloud. Type the request in a text editor, not on paper: in Lab 1 you paste it into Genie Code. Use the six ingredients:
   - **Business intent:** what the result is for, and who uses it
   - **Input schema and grain:** the tables, what one row is, the keys, and what nothing guarantees
   - **Expected result:** columns, types, and grain of the output, and where every other row goes
   - **Permitted libraries:** the language and the API, and what must not be added
   - **Error behavior:** what happens to a NULL, an unknown key, a duplicate key, an unexpected value
   - **Acceptance criteria:** the checks that you will run, with numbers where you have them

   Two habits belong to every request: ask for the assumptions first, and ask for one small change at a time.
2. **Challenge, 2 minutes.** The reviewer names two things, and the driver writes them down: lines 1 and 2 of the box. Then write lines 3 and 4 together.
3. **Checkpoint, 1 minute.** Your instructor asks two pairs for lines 3 and 4. Be ready to say both sentences.

RECORD YOUR EVIDENCE

1 One business rule that our request does not state: _____

2 One reason not to accept an answer to our request immediately: _____

3 A correct result means: _____

4 A reviewer will ask us for this evidence: _____

CHECKPOINT

Your request names the grain of the input, says where the rows go that are not accepted, and ends with at least two checks that a result must pass. Keep the editor window open.

WATCH FOR

A request is not better because it is longer. Every sentence must close a gap that the assistant would otherwise fill with an assumption. "Write clean, efficient, well-documented code" closes none.

Lab 1: Build the first reviewed transformation

Module 2 · 20 minutes · in pairs · Genie Code in an Azure Databricks notebook

You build the accepted invoice dataset in three small steps. After each step you review the proposal before it runs, run it yourself, and let a supplied acceptance check judge the result.

YOU PRACTICE	YOU START FROM	YOU FINISH WITH
A bounded request, a review before execution, and a check after it	The notebook `Lab1_Build_the_first_reviewed_transformation`, and your request from Exercise 1	Seven passing checks on two datasets, three recorded assumptions, and a list of open test targets

What you build

Scope: rules R2, R5, and R6 of the rule sheet, and the derived amounts. The other rules stay open on purpose, and check C6 tells you which.

The pattern has three parts, and it is the pattern of the whole course: **facts first, one decision, split last.** No step before the split filters, drops, or multiplies a delivered row. That is why every check can ask the same question: is every row still there?

Minutes	Part	You do	Checks
0 to 3	Get ready	Run three supplied cells, read the facts, give Genie Code your context	
3 to 7	Task 1	Keep every delivered row and add three facts: status class, tax amount, gross amount	C1, C2, C3
7 to 11	Task 2	Count the master rows of every customer key and every product key	C4
11 to 16	Task 3	Decide one outcome for every row. Then split accepted lines and exceptions.	C5, C6, C7
16 to 19	Prove it	Run everything on the training data	7 of 7
19 to 20	Record	Three assumptions, one expression that you can explain, the open test targets	

Get ready

1. Open the notebook `Lab1_Build_the_first_reviewed_transformation` and attach it to the compute of your class. The widget **Dataset** at the top says `fixture`: 56 delivered rows in 48 named cases.
2. Run the first three code cells, one after the other: `SETUP`, `ACCEPTANCE CHECKS`, and `FACTS`. They are supplied, and you do not change them.

```
1. The status values as delivered. How many spellings mean 'posted'?
     'POSTED'            47
     'DRAFT'              3
     '   '                1
     ' posted'            1
     'CANCELLED'          1
     'Cancelled'          1
     'PENDING'            1
     'Posted '            1
```

```
2. Master keys with more than one row. What does a join do with them?
     customers: 1 keys have more than one row
     products: 1 keys have more than one row

3. Signs. Is a negative amount an error?
     5 delivered rows have a negative Quantity: credit lines
     3 delivered rows have no TaxRate: no tax applies to them
```

Read the three facts with your partner. An assistant that does not know them will guess.

3. Open the Genie Code pane: the sparkle icon in the upper-right corner of the page. Check the setting once more: **More** > **Settings** > **Actions** must say **Ask first**.
4. Select **New chat**. Paste your request from Exercise 1, and end it with these three lines:

```
In this notebook I will ask for one small step at a time. Change only the cell that I name.
Do not run anything unless I ask. Do not write code yet.
First list the assumptions that you would have to make about this data, and wait for my answer.
```

5. Read the assumptions that come back. Answer two of them from the rule sheet or from the facts. You record the three most important ones at the end of the lab.

IF IT DOES NOT WORK

Genie Code is slow or not available: open `prepared_ai_outputs/Lab1_prepared_response_Genie_Code.md` from your package. It holds one response for each task, in the style of the assistant. The three responses contain five problems on purpose. Review them like any proposal, paste the code into the task cell, and let the checks tell you what you missed.

How every task runs

Every task has the same five moves. The reviewer of the pair leads moves 2 and 5.

1. **Request.** Send the request to Genie Code.
2. **Review before it runs.** Genie Code asks for permission before it changes the notebook: read what it wants to do, and select **Allow** for the edit of the task cell. If it asks to run code, select **Skip**: in this lab you run every cell yourself. Then read the changed cell and answer the three review questions aloud.
3. **Run the task cell** with `Shift+Enter`.
4. **Run the check cell** below it.
5. **Read the check.** `PASS` means that this step holds. `FAIL` comes with the row that proves it: look the row up, repair the cell, and run both cells again.

WATCH FOR

Do not ask Genie Code to make a failing check pass. Tell it what the check reported and which rule the code breaks, and ask for the smallest correction of the task cell. The check cells are supplied and stay as they are.

Task 1. Three facts for every delivered row

Request

```
Complete only the cell that starts with "# TASK 1". Keep every row of `lines` and every delivered
column unchanged, and add three columns:
- StatusClass: 'POSTED' when Status, trimmed and in upper case, is POSTED. 'NOT_POSTED' when it is
  DRAFT or CANCELLED. 'UNEXPECTED' for any other value, including NULL.
- TaxAmount: NetAmount x TaxRate, rounded half up to 2 decimals, as DECIMAL(18,2).
  When TaxRate is NULL, no tax applies: 0.00.
- GrossAmount: NetAmount + TaxAmount, as DECIMAL(18,2).
Do not filter. Do not overwrite Status. Do not run the cell: I review it first.
```

Review before you run

1. Does any line filter, drop, or deduplicate rows?
2. Is a delivered column overwritten, for example `Status`?
3. Is an amount cast to `double` or `float`? What happens when `TaxRate` is NULL?

Expect

```
PASS  C1  Every delivered row is still there, once, with its delivered values
          56 delivered rows, 56 rows in prepared
          delivered values are unchanged
PASS  C2  StatusClass follows rule R2, compared with the rule in SQL
          StatusClass: POSTED 49, NOT_POSTED 5, UNEXPECTED 2
PASS  C3  TaxAmount and GrossAmount: decimal, rounded half up, signs kept
          4 credit lines keep negative amounts
          22 of 22 hand calculations of the fixture match
```

Task 2. Count the master rows of every key

Write this request yourselves. The result is a DataFrame named `matched`: every row of `prepared`, plus two columns. `CustomerMatches` is the number of rows in `customers` with exactly this `CustomerID`, and `ProductMatches` is the same for `products` and `ProductID`.

Your request says:

- which cell changes, and that nothing else changes
- the two new columns, and what they hold when no master row matches
- the grain of the output: exactly one row for each row of `prepared`
- what the assistant must not do: join the master attributes now, or trim a key, or change its case
- that you review before anything runs

Review before you run

1. Which join type is used, and what is joined: the master table, or one row for each key?
2. Can a key with two master rows produce two output rows?
3. What does a key without a master row get: 0, NULL, or no row at all?

Expect

```
PASS  C4  Master matches: no row lost, no row multiplied, exact keys
          56 delivered rows, 56 rows in matched
          customer keys: rows without a master row 7, rows with more than one master row 1. Unmatched keys,
              for example: NULL, 'C299999', 'c200001 '
          product keys: rows without a master row 4, rows with more than one master row 1. Unmatched keys,
              for example: NULL, 'P-29999'
```

IF IT DOES NOT WORK

Stuck for more than two minutes: use the reference request. It is also in `prompts/LAB_PROMPTS.md` .

```
Complete only the cell that starts with "# TASK 2". Add two columns to `prepared` and name the
result `matched`:
- CustomerMatches: the number of rows in `customers` with exactly this CustomerID. 0 when there is
    none.
- ProductMatches: the number of rows in `products` with exactly this ProductID. 0 when there is none.
`matched` must have exactly one row for each row of `prepared`. Aggregate each master table to one
row for each key before you join it. Do not join the master attributes yet. Do not trim a key and
do not change its case. Do not run the cell: I review it first.
```

Task 3. One outcome for every row, then two outputs

Switch driver and reviewer now.

Request

```
Complete only the cell that starts with "# TASK 3".
1. Add ReasonCode to `matched`. The first rule that fails decides: NOT_POSTED when StatusClass is
   NOT_POSTED. UNEXPECTED_STATUS when StatusClass is UNEXPECTED. UNKNOWN_CUSTOMER when CustomerMatches
   is 0, AMBIGUOUS_CUSTOMER when it is above 1. UNKNOWN_PRODUCT and AMBIGUOUS_PRODUCT in the same way
   for ProductMatches. Otherwise NULL.
2. Add Disposition: ACCEPTED when ReasonCode is NULL, EXCLUDED for NOT_POSTED, QUARANTINED for every
   other reason.
3. `accepted`: the ACCEPTED rows, joined to `customers` for CustomerName, Segment, Country, Region,
   and to `products` for ProductName, ProductLine.
4. `exceptions`: every other row. No row may be dropped, and no missing value may be filled.
Do not select or rename columns: the next cell does that. Do not run the cell: I review it first.
```

Review before you run

1. Is the order of the conditions the order of the rules? What does a draft with an unknown customer get?
2. Why can the joins to `customers` and `products` not multiply a row here, although they could in Task 2?
3. Where do the rows go that are not accepted? Is any value replaced by a placeholder such as 'UNKNOWN'?

Expect

```
PASS  C5  Every delivered row is in exactly one output
          56 delivered rows, 56 rows in accepted + exceptions: accepted 38 + exceptions 18
PASS  C6  One outcome for each row, in the order of the rules
          rows by outcome: (accepted) 38, AMBIGUOUS_CUSTOMER 1, AMBIGUOUS_PRODUCT 1, NOT_POSTED 5,
              UNEXPECTED_STATUS 2, UNKNOWN_CUSTOMER 7, UNKNOWN_PRODUCT 2
          fixture, contract 1.0: 35 of 35 rows that this lab can decide have the expected outcome
          OPEN TEST TARGETS: 21 fixture rows need a rule that this lab does not implement: R1 required
              fields 5, R3 line identity 9, R4 invoice date 4, R7 amount 3
                              16 of them are accepted by your code today. They are the test targets of Lab
                                 3.
PASS  C7  The accepted output: contract columns, one master row, validated totals
          38 accepted lines, 4 of them credit lines
          on the training data this check also compares your totals with the validated totals, for each
              currency
LAB 1 CHECKS on dataset 'fixture': 7 of 7 passed
```

CHECKPOINT

`LAB 1 CHECKS on dataset 'fixture': 7 of 7 passed`. Read the two lines under `OPEN TEST TARGETS`: 21 fixture rows need a rule that this lab does not implement, and your code accepts 16 of them today. Seven green checks do not make the dataset correct. They make this scope correct.

Prove it on the training data

Seven checks passed on 56 rows. Now the same code meets 49,700 rows.

1. Set the widget **Dataset** at the top of the notebook to `training`.
2. Select **Run all**.
3. Compare the lines of checks C5, C6, and C7 with this output.

```
PASS  C5  Every delivered row is in exactly one output
          49,700 delivered rows, 49,700 rows in accepted + exceptions: accepted 45,510 + exceptions 4,190
PASS  C6  One outcome for each row, in the order of the rules
          rows by outcome: (accepted) 45,510, AMBIGUOUS_CUSTOMER 332, AMBIGUOUS_PRODUCT 182, NOT_POSTED
              3,050, UNEXPECTED_STATUS 86, UNKNOWN_CUSTOMER 262, UNKNOWN_PRODUCT 278
PASS  C7  The accepted output: contract columns, one master row, validated totals
          45,510 accepted lines, 1,692 of them credit lines
          validated totals of the Lab 1 scope: 6 of 6 currencies match in rows, net, and gross
LAB 1 CHECKS on dataset 'training': 7 of 7 passed
```

CHECKPOINT

`7 of 7 passed`, the numbers of checks C5 and C6 are the numbers above, and check C7 says that 6 of 6 currencies match the validated totals in rows, net amount, and gross amount. The validated totals were calculated independently of your code. That comparison is the difference between "it ran" and "it is right".

The cell `RESULT` shows what your exception report would tell the business today: 4,190 delivered rows that are not revenue, each with a reason.

Record

Double-click the last cell of the notebook, "Record", and fill it in. The discussion starts there.

RECORD YOUR EVIDENCE

Three assumptions that we confirmed or corrected:

One generated expression that we can explain without the assistant:

How our joins and filters behave, in two sentences:

Open test targets, from check C6 on the fixture:

Checkpoint

Be ready to do both when your instructor asks:

1. **Explain one generated expression** without the assistant: what it does to a NULL, to a credit line, or to a key with two master rows.
2. **Show a result that agrees with the fixture:** the line of check C6 that says how many expected outcomes match.

The rows that your code cannot decide yet are not an ignored warning. They are a documented test target, and Lab 3 starts with them.

If something does not work

You see	Cause	Do this
A check says that a column or a DataFrame does not exist yet	The task cell was not run, or the result has another name	Run the task cell. The names `prepared`, `matched`, `decided`, `accepted`, and `exceptions` are fixed.
Genie Code changed another cell, or added cells	The request did not name the cell, or the assistant ignored it	Undo the edit with `Ctrl+Z` in the cell, or delete the new cell. Repeat: "Change only the cell that starts with # TASK 1."
Genie Code ran the cell before you read it	The approval request was allowed without reading, or **Actions** is not **Ask first**	Check the setting. Nothing is damaged: the lab writes no table. Read the cell now.
C2 fails with a row such as `'Posted '`	The status is compared without `trim`, or without `upper`	Normalize inside the comparison only. `Status` itself stays as delivered.
C3 fails for rows without a tax rate	`NULL` times a number is `NULL`	`coalesce` the rounded tax to 0. Do not fill `TaxRate`.
C4 says "yours says 1" for a key such as `'c200001 '`	The keys were trimmed or changed in case before the count	Rule R5: a key matches only when it is exactly equal
C5 says that rows are missing	`exceptions` holds only some of the rows that are not accepted	`exceptions` is every row whose `Disposition` is not `ACCEPTED`

You see	Cause	Do this
An error names an ambiguous column in Task 3	A join condition compares two columns of the same name	Join on the column name: `.join(customers.select(...), "CustomerID", "left")`
The numbers differ after you changed the widget	The `SETUP` cell did not run again	**Run all**

> **IF YOU ARE AHEAD**
>
> Open `02I_Fixture_Workbook.xlsx` and look up the cases that check C6 lists as open: which rule decides each of them, and what is the expected outcome? Then ask Genie Code, with `/explain`, to explain one line of your Task 3 cell, and compare its explanation with your own.
>
> **A design question.** Task 2 counts the master rows before the join. What would you gain, and what would you risk, if you joined first and counted afterwards?

Lab 2: Refactor, test, run, compare

Module 3 · 22 minutes · in pairs · GitHub Copilot in Visual Studio Code, runs on Azure Databricks

The logic of Lab 1 works, and it lives in one long function that no test can call. You move it into a module with GitHub Copilot, test what can be tested on your workstation, run the same revision on Databricks, and prove that nothing changed its meaning on the way.

YOU PRACTICE	YOU START FROM	YOU FINISH WITH
A scoped edit across files, a review of the difference, local tests, and a parity check	The project `o2i-workshop` at the checkpoint `lab2-start`: 4 passed, 8 failed	A module, 12 passed, 0 failed, and `PARITY: ALL COMPARISONS PASS` for one revision

The project in two minutes

So that every pair starts from the same code, the project holds a reviewed version of the Lab 1 logic. It is correct for the scope of Lab 1.

File	What it is	In this lab
`scratch/lab1_reviewed.py`	The Lab 1 logic in one function, as it came out of the notebook	The reference. You never edit it.
`o2i/transform.py`	The same logic as small PySpark functions. Three of them raise `NotImplementedError`.	You complete `count_master_matches`, `decide`, and `with_amounts`
`o2i/rules.py`	The business rules in plain Python: no Spark, standard library only. Two functions raise `NotImplementedError`.	You complete `tax_amount` and `gross_amount`
`tests/unit/`	Code tests for `o2i/rules.py`. They run on your workstation in milliseconds.	They decide step 3
`run_transform.py`, `run_parity.py`	Thin entry points without business logic. They run on Databricks.	They decide steps 2 and 4
`AGENTS.md`	The instructions of this project for an assistant: layout, nine hard rules, how to work	The assistant reads it by itself. Read it once yourself.
`prompts/LAB_PROMPTS.md`	Every request of Labs 1 to 4, ready to paste	Open it in a second editor tab

Two things run in two places. The code tests are plain Python and run on your workstation. Everything that touches a table runs on Databricks compute, whichever way you start it.

Minutes	Step	You do	Decided by
0 to 2	Get ready	New chat, four controls, code tests	`4 passed, 8 failed`
2 to 5	1. Orient	Ask what moves where. No edit.	Your reading of the answer
5 to 11	2. Move the logic	Three functions in `o2i/transform.py`. Review the difference. Run on Databricks.	The outcome table of the run
11 to 16	3. Write the twins	Two functions in `o2i/rules.py`. Run the code tests.	`12 passed, 0 failed`
16 to 20	4. Compare	Run the parity check on Databricks	`PARITY: ALL COMPARISONS PASS`
20 to 22	5. Checkpoint	Name what ran where, for which revision	Two equal fingerprints

Get ready

1. In VS Code, open the project folder `o2i-workshop`. In the terminal, check where you are:

```
python tools/checkpoint.py list
```

```
* lab2-start
  lab3-start
  lab4-start
* = loaded last
```

2. Run the code tests:

```
python tools/run_tests.py
```

> **CHECKPOINT**
> The last line is `4 passed, 8 failed`. The four tests of the status rule pass. The eight failing tests call `tax_amount` and `gross_amount`, which are not written yet.

3. Open the Chat view with `Ctrl+Alt+I` and start a new chat with **New Chat (+)**. Set the four controls of the chat input as in "The four controls of the chat" in "Before you start".

> **IF IT DOES NOT WORK**
> GitHub Copilot is not available: run `python tools/checkpoint.py prepared lab2`. It writes a prepared change over your files, in the style of the assistant, for steps 2 and 3. It contains four problems on purpose. Review it with `python tools/checkpoint.py diff`, then continue with the runs of steps 2 to 4 and repair what they report. `prepared_ai_outputs/Lab2_prepared_change_Copilot.md` has the response that belongs to the change.

Step 1. Orient, without an edit

Send request 2.1. For a question, use **Ask** where your chat offers it. The request itself also says that nothing is edited.

```
Read AGENTS.md, scratch/lab1_reviewed.py, o2i/transform.py, and o2i/rules.py. Do not edit any file.
1. Three functions in o2i/transform.py and two functions in o2i/rules.py raise NotImplementedError.
   For each of them, name the lines of scratch/lab1_reviewed.py that hold its logic.
2. List everything in scratch/lab1_reviewed.py that you would be tempted to improve while you move
   it. I want the logic moved, not improved.
3. List the assumptions that you make about the data.
```

Read the answer with your partner, and keep the second list. It is the list of changes that you will decline in step 2.

> **WATCH FOR**
> If the answer does not quote the files, the assistant has not read them. Add them as context: type `#` in the chat input and choose each file, or drag the files from the Explorer into the chat.

Step 2. Move the logic

1. Switch the **Agent** control to **Agent** and send request 2.2.

   ```
   Complete count_master_matches, decide, and with_amounts in o2i/transform.py. Move the logic from
   scratch/lab1_reviewed.py without changing its meaning.
   - Keep every function signature and every docstring.
   - Use the names of o2i/config.py where the module already uses them.
   - Edit only o2i/transform.py. Do not touch scratch/, tests/, docs/, or any other file.
   - No new import and no new dependency. No pandas, no UDF, no cache, no persist.
   - Do not run anything. I run the tests myself.
   After the edit, list each function that you changed and the lines of lab1_reviewed.py that it came from.
   ```

2. **Review the difference before you keep it.** Open `o2i/transform.py`. The reviewer answers aloud:
   1. Which files changed? Only `o2i/transform.py` may be in the list.
   2. Is there a new import at the top of the file, or a call that the request ruled out?
   3. Compare three places with `scratch/lab1_reviewed.py`: the two `coalesce` calls of the counts, the order of the `when` conditions of the reason code, and the `coalesce` around the rounded tax. Is each of them still there, with the same meaning?

   In a Local session, select **Keep** when all three answers are right, and **Undo** when one is not. In a Copilot session the edit is already saved: when an answer is wrong, restore the checkpoint of your request and ask again. On both, this command shows every difference from the start of the lab:

   ```
   python tools/checkpoint.py diff
   ```

3. **Run it where the data is.** Open `run_transform.py` and run it on Databricks, with the run route of your class.

   ```
   o2i revision <your revision> | contract 1.0, rules R2, R5, R6 only | Spark <version>
   input main.o2i | output main.o2i_<your name>

   Dataset 'fixture': 56 rows written to main.o2i_<your name>.fixture_accepted and main.o2i_<your
       name>.fixture_exceptions
     ACCEPTED                                  38
     EXCLUDED     NOT_POSTED                    5
     QUARANTINED  AMBIGUOUS_CUSTOMER            1
   ```

```
  QUARANTINED  AMBIGUOUS_PRODUCT               1
  QUARANTINED  UNEXPECTED_STATUS               2
  QUARANTINED  UNKNOWN_CUSTOMER                7
  QUARANTINED  UNKNOWN_PRODUCT                 2

Dataset 'training': 49,700 rows written to main.o2i_<your name>.training_accepted and main.o2i_<your
    name>.training_exceptions
  ACCEPTED                                45,510
  EXCLUDED     NOT_POSTED                  3,050
  QUARANTINED  AMBIGUOUS_CUSTOMER            332
  QUARANTINED  AMBIGUOUS_PRODUCT             182
  QUARANTINED  UNEXPECTED_STATUS              86
  QUARANTINED  UNKNOWN_CUSTOMER              262
  QUARANTINED  UNKNOWN_PRODUCT               278
```

CHECKPOINT

Both outcome tables are the tables above, to the row. They are also the numbers of your Lab 1 notebook: 45,510 accepted lines on the training data.

WATCH FOR

An assistant that is asked to move code likes to improve it: a shorter expression, a "safer" default, a join that also fetches the attributes. Every one of these changes the meaning somewhere. Decline it, and say why in one sentence. The parity check of step 4 will find what you let through.

Step 3. Write the plain-Python twins

Switch driver and reviewer now.

The Spark expressions cannot be tested without Spark. Their twins in `o2i/rules.py` can: Decimal in, Decimal out, in milliseconds.

1. Send request 2.3.

```
Write tax_amount and gross_amount in o2i/rules.py. They are the plain-Python twins of the Spark
expressions in with_amounts in o2i/transform.py.
- Decimal in, Decimal out. Pass every argument through _as_decimal, so that a float raises TypeError.
- Round half up to two decimals with round_half_up.
- A tax rate of None means that no tax applies: the tax is 0.00. A tax rate of 0 is a rate.
- A credit stays negative.
Edit only o2i/rules.py. Do not edit the tests. Do not run anything.
Then tell me, without running code, what tax_amount(Decimal("3.00"), Decimal("0.0550")) and
tax_amount(Decimal("-3.00"), Decimal("0.0550")) return, and why.
```

2. **Review.** The two predictions at the end of the answer must be 0.17 and -0.17: 3.00 × 0.0550 is exactly 0.165, and half up means away from zero. An answer of 0.16 means that the code, or the assistant, rounds half to even. Then read the two functions: is there a `float`, a `round()` without the helper, or a tax rate of `None` that reaches a multiplication?
3. Keep the edit, and run the code tests yourself:

```
python tools/run_tests.py
```

> **CHECKPOINT**
> The last line is `12 passed, 0 failed`. If a test fails, read its name and its message before you ask the assistant: the name says which sentence of the rule sheet is broken. The tests are the specification. They stay as they are.

Step 4. Compare: parity is more than a total

You now have two implementations of the same logic: the reference in `scratch/` and your module. `run_parity.py` runs both on Databricks and compares their outputs in five ways.

1. Open `run_parity.py` and run it on Databricks.

```
o2i revision <your revision> | contract 1.0, rules R2, R5, R6 only | Spark <version>
input main.o2i | output main.o2i_<your name>

Parity of accepted lines, dataset 'fixture'
  PASS  schemas         columns: 23, same names and types
  PASS  key sets        keys only in the reference: 0; keys only in the candidate: 0
  PASS  row counts      reference 38 rows and 38 keys, candidate 38 rows and 38 keys
  PASS  signed amounts  rows that differ in NetAmount, TaxAmount, GrossAmount: 0
  PASS  aggregates      groups by Currency: 5, all equal
...
PARITY: ALL COMPARISONS PASS
```

Comparison	It finds
Schemas	A column that changed its name or its type, for example money that became a `double`
Key sets	A row that exists on one side only. The key is `RawRowID`, the delivered row.
Row counts	A line that was lost, or written twice
Signed amounts	An amount that differs in a row, even when the totals agree
Aggregates	A row that moved to another group, even when the grand total agrees
Rules agreement, on the fixture	A difference between the Spark expression and its plain-Python twin

> **CHECKPOINT**
> The last line is `PARITY: ALL COMPARISONS PASS`. A line that says `FAIL` names what differs: fix your module, never the reference, and run the comparison again.

2. **Tie both sides to one revision.** In the terminal:

```
python tools/revision.py
```

The command prints a fingerprint of your `o2i` package: twelve characters that change when any file of the package changes. Every run on Databricks prints its fingerprint in the line `o2i revision`.

> **CHECKPOINT**
> Both fingerprints are equal: the code that you reviewed is the code that produced the parity result. The pair next to you has another fingerprint: it identifies the code, not the lab.

Step 5. Checkpoint

RECORD YOUR EVIDENCE

What ran on our workstation, and what it proved: _____

What ran on Databricks, and what it proved: _____

Fingerprint on the workstation: _____

Fingerprint printed by the parity run: _____

One change that we declined, and why: _____

KEY MESSAGE

Matching a grand total is not proof of equivalence. Two results can share a total and differ in rows, keys, or signs. Parity means that schemas, key sets, row counts, signed amounts, and aggregates all agree, for one revision.

If something does not work

You see	Cause	Do this
`NotImplementedError: Lab 2: ...` in a run on Databricks	One of the three functions is not moved yet	Read the function name in the last lines of the error, and go back to step 2
The fingerprint of the run differs from `python tools/revision.py`	A file was not saved, or the run used an older copy	Save all files, and run again. On run route D, the files in the workspace folder are the ones that count.
`FAIL signed amounts` or `FAIL rules agreement`	The tax expression lost its `coalesce`, uses another rounding, or changed a type	Compare `with_amounts` with the last lines of the reference, character by character
`FAIL aggregates` on the exceptions	A reason code changed: the order of the conditions, or a missing condition	Compare `decide` with the reference. The first rule that fails decides.
The assistant edited a test, or a file in `scratch/` or `docs/`	It solved another problem than yours	Undo the edit. `AGENTS.md` forbids it: say so, and repeat the request.
The assistant asks to run a command	It wants to check its own work	`python tools/run_tests.py` is harmless: allow it once. Decline anything that installs, deletes, or downloads.
A run on Databricks does not start	The compute is stopped, or the sign-in expired	Start the compute in the Databricks view, or sign in again. Still nothing: raise your hand.

IF YOU ARE AHEAD

Run `demo/parity_trap.py` on Databricks. Four candidates share the EUR grand total of the reference, and none of them is equal to it. For each candidate, say which change in real code would produce it. Then open `prepared_ai_outputs/Lab2_prepared_change_Copilot.md` and find its four problems by reading only. **A design question:** the rules now exist twice, in Spark and in plain Python. What does the second copy cost, and what keeps the two from drifting apart?

Lab 3: Make the failure visible, then repair it

Module 4 · 30 minutes · in pairs · GitHub Copilot in Visual Studio Code, runs on Azure Databricks

A colleague has extended your module to all seven rules of the contract. The code tests are green and the run finishes without an error. Three defects are in the code. You write the tests and checks that make each of them visible, trace each failure to its cause, and only then ask for a fix.

YOU PRACTICE	YOU START FROM	YOU FINISH WITH
Test cases from the contract, checks that report record keys, tracing a failure, and judging a fix	The checkpoint `lab3-start`: 26 passed, 0 failed, 3 skipped	Two code tests, three data-quality checks, before-and-after evidence for three causes, and `REGRESSION: ALL GREEN`

Three rules of this lab

1. **Never delete, skip, or weaken a failing assertion** to get a passing run.
2. **The assistant does not define what is correct.** It must not invent an expected total, and it must not redefine a business rule. Expected values come from the rule sheet, calculated by hand.
3. **Always compare with the fixture**: 56 rows that you can verify by hand, with expected outcomes that did not come from the code.

Two kinds of evidence

	Code tests	Data-quality checks
Question	Does a function behave as specified?	Do the records, the relationships, and the outputs satisfy the contract?
Where	`tests/unit/`, plain Python	`o2i/checks.py`, PySpark, on the written output tables
Runs	On your workstation, in milliseconds: `python tools/run_tests.py`	On Databricks: `run_checks.py`
Cannot see	Anything that happens to a table: a lost row, a join, a filter	A function that no row of the data exercises

`run_all.py` is the full regression suite: the code tests, then the transformation and every check, on every dataset of `lab_config.DATASETS`.

Minutes	Step	You do	Decided by
0 to 4	Get ready	Load the checkpoint, run the tests, run the checks	Green, and wrong
4 to 8	1. Find the missing cases	Ask for test cases from the contract. No edit.	Your hand calculation
8 to 12	2. Two code tests	Write them, run them, see them fail	`26 passed, 2 failed, 1 skipped`

Minutes	Step	You do	Decided by
12 to 18	3. Three data-quality checks	Write them, review them, run them before any fix	passed 6, failed 5, skipped 1 on the fixture
18 to 22	4. Trace	One failure to records, one failure to a decision in the code	Rows and one expression
22 to 28	5. Repair	The smallest fix for each cause, one at a time, then the full suite	`REGRESSION: ALL GREEN`
28 to 30	6. Checkpoint	Before-and-after evidence, and one sentence for each fix	Your explanation

Get ready

1. Load the checkpoint of the lab. Your Lab 2 files are copied to the folder `.backups` first.

```
python tools/checkpoint.py lab3-start
```

```
Loaded checkpoint 'lab3-start': 26 files. Your previous files are in .backups/<date>-<time>-before-lab3-
    start
Next: python tools/run_tests.py
```

With Git on your workstation, one more line says that the checkpoint was committed. The Source Control view then shows only your own changes.

2. Run the code tests.

```
python tools/run_tests.py
```

> **CHECKPOINT**
>
> `26 passed, 0 failed, 3 skipped`. Nothing fails. Three tests are placeholders and are skipped. **Skipped is not passed.**

3. Run `run_checks.py` on Databricks. `lab_config.py` says `DATASETS = ["fixture", "training"]`. This is the block of the training data:

```
Dataset 'training': 49,468 rows written to main.o2i_<your name>.training_accepted and main.o2i_<your
    name>.training_exceptions
  ACCEPTED                              43,508
  EXCLUDED     NOT_POSTED                3,042
  QUARANTINED  AMBIGUOUS_CUSTOMER          332
  QUARANTINED  AMBIGUOUS_PRODUCT           181
  QUARANTINED  AMOUNT_MISMATCH            1,698
  QUARANTINED  INVALID_DATE                 48
  QUARANTINED  MISSING_REQUIRED_FIELD       60
  QUARANTINED  UNEXPECTED_STATUS            80
  QUARANTINED  UNKNOWN_CUSTOMER            248
  QUARANTINED  UNKNOWN_PRODUCT             271
Data-quality checks on dataset 'training'
  PASS  schema_contract           columns and types match the contract
  PASS  required_fields           no NULL in a required output column
  SKIP  reconciliation            TODO: reconciliation is not written yet (Lab 3, check 1)
  SKIP  amount_totals             TODO: amount_totals is not written yet (Lab 3, optional check 4)
```

```
  PASS  unique_grain               one accepted row for each InvoiceID and LineNumber
  SKIP  conflicts_held             TODO: conflicts_held is not written yet (Lab 3, check 2)
  PASS  referential_integrity      every accepted key has exactly one master row
  PASS  status_rule                only posted lines accepted, no posted line turned away for its status
  SKIP  signed_amounts             TODO: signed_amounts is not written yet (Lab 3, check 3)
  SKIP  expected_outcomes          this dataset has no expected results
  SKIP  expected_amounts           this dataset has no expected amounts
  SKIP  spec_agreement             runs on the fixture only: it collects every row
  passed 5, failed 0, skipped 7
```

CHECKPOINT

On the training data no check fails: `passed 5, failed 0, skipped 7`. Now read the first line of the block: 49,468 rows were written, and 49,700 rows were delivered. No check that exists today asks where the others went. Only the fixture objects: on the fixture, 18 of 56 rows differ from their expected outcome.

4. Start a new chat, and set the four controls.

WATCH FOR

Running on the training data takes longer than running on the fixture. While you write and repair, set `DATASETS = ["fixture"]` in `lab_config.py`: a run then takes seconds. The last line of a run counts the failed checks of all datasets of that run, so it changes with this setting. For the last run of the lab, set both datasets again.

Step 1. Find the cases that no test covers

Send request 3.1. It edits nothing.

```
Read docs/business_contract.md, tests/unit/test_rules.py, and tests/unit/test_fixture.py.
Do not edit any file. Do not take any expected value from o2i/rules.py or o2i/transform.py.
List up to eight cases of the business contract that no code test covers yet. For each case give:
the rule or section of the contract, concrete input values, the expected result, and how you
derived the expected result from the contract.
Mark every expected result that you calculated yourself with CHECK BY HAND.
```

Decide for each proposed case: accept, reject, or already covered. Then do what the mark says. The contract states that rounding is half up, and no test uses a value that ends exactly on half a cent:

RECORD YOUR EVIDENCE

3.00 × 0.0550 = 0.165. Half up to two decimals: _____

-3.00 × 0.0550 = -0.165. Half up to two decimals: _____

The fixture workbook, cases F04, F06, and F08, shows the same kind of value. Our numbers agree with it: yes / no _____

IF IT DOES NOT WORK

No assistant: `prepared_ai_outputs/Lab3_prepared_test_cases.md` holds a prepared list of cases. Not every case in it is correct, and not every expected value is right. Decide with the rule sheet.

Step 2. Two code tests, and they must fail

1. Send request 3.2. The expected values in it are the ones that you just verified.

```
In tests/unit/test_rules.py, replace the placeholder bodies of test_tax_rounds_half_up_on_a_tie and
test_tax_of_a_credit_rounds_away_from_zero_on_a_tie with real tests.
Use these cases. I verified them by hand against the contract and the fixture workbook:
- net 3.00 at rate 0.0550: tax 0.17, gross 3.17
- net 0.50 at rate 0.2100: tax 0.11
- net -3.00 at rate 0.0550: tax -0.17, gross -3.17
Write the expected values as Decimal literals. Do not compute an expected value with the function
under test. Edit only these two tests. Do not edit anything in o2i/. Do not run anything.
```

2. **Review.** Every expected value is a literal such as `D("0.17")`. A test that calculates its expected value with `round()`, or with the function under test, repeats the defect that it should find. Nothing outside the two tests changed.
3. Run the code tests.

```
FAIL: test_tax_of_a_credit_rounds_away_from_zero_on_a_tie
    (tests.unit.test_rules.AmountTests.test_tax_of_a_credit_rounds_away_from_zero_on_a_tie)
AssertionError: Decimal('-0.16') != Decimal('-0.17')
FAIL: test_tax_rounds_half_up_on_a_tie
    (tests.unit.test_rules.AmountTests.test_tax_rounds_half_up_on_a_tie)
AssertionError: Decimal('0.16') != Decimal('0.17')
Ran 29 tests in 0.0XXs
FAILED (failures=2, skipped=1)
26 passed, 2 failed, 1 skipped
```

> **CHECKPOINT**
>
> `26 passed, 2 failed, 1 skipped`. Both new tests fail, and the message shows the defect: the code returns 0.16 where the contract says 0.17. A new test that fails for the reason that you predicted is your first piece of evidence. Do not repair anything yet.

Step 3. Three data-quality checks, run before any fix

Switch driver and reviewer now.

Three checks in `o2i/checks.py` are placeholders. Each docstring is the specification of its check.

Check	It asks	Test matrix
`reconciliation`	Is every delivered row in exactly one output, counted by `RawRowID`?	Reconciliation
`conflicts_held`	Is every key that was delivered with different content held, and none of its rows accepted?	Uniqueness and replay
`signed_amounts`	Do the accepted amounts keep sign and value, and is no valid credit held as a mismatch?	Business rules

1. Send request 3.3.

```
In o2i/checks.py, implement the three checks whose docstrings say "LAB 3, CHECK 1", "LAB 3, CHECK 2",
and "LAB 3, CHECK 3": reconciliation, conflicts_held, and signed_amounts. Each docstring is the
specification. Keep the docstrings.
- Follow the style of the checks that are already implemented in this file: read from ctx, return
  a CheckResult, and use _keys for the evidence.
- A check reports. It never changes data, and it never filters the delivered rows to make itself pass.
- PySpark DataFrame API only. Do not collect a full table. No pandas and no UDF.
Edit only these three functions. Do not edit o2i/transform.py or o2i/rules.py. Do not run anything.
```

2. **Review each check before you keep it.**
   1. What does it count: delivered rows by `RawRowID`, or something that is easier to make equal?
   2. Does it name records? A check that says "3 rows differ" and names no row cannot be traced.
   3. Does it leave rows out before it counts, for example with `distinct`, `dropDuplicates`, or a filter that the docstring does not ask for?
3. Run `run_checks.py` on Databricks. This is the block of the fixture:

```
Data-quality checks on dataset 'fixture'
  PASS  schema_contract            columns and types match the contract
  PASS  required_fields            no NULL in a required output column
  FAIL  reconciliation             raw 56, accepted 21 + exceptions 27 = 48; rows in no output: 8; rows
      written more than once: 0
          in no output: F0017
          in no output: F0018
          in no output: F0019
          in no output: F0020
          in no output: F0021
  SKIP  amount_totals              TODO: amount_totals is not written yet (Lab 3, optional check 4)
  PASS  unique_grain               one accepted row for each InvoiceID and LineNumber
  FAIL  conflicts_held             conflicting keys delivered: 2; accepted rows of these keys: 2
          F0055 / INV-3000018 / 1
          F0056 / INV-3000019 / 1
  PASS  referential_integrity      every accepted key has exactly one master row
  PASS  status_rule                only posted lines accepted, no posted line turned away for its status
  FAIL  signed_amounts             accepted rows with mixed signs: 1; changed amounts: 0; matching
      amounts held as AMOUNT_MISMATCH: 4; accepted credit lines: 0
          accepted with mixed signs: F0033 / -2.000 / 183.58
          held as AMOUNT_MISMATCH although the amount matches: F0005 / -4.000 / 91.7875 / -367.15
          held as AMOUNT_MISMATCH although the amount matches: F0006 / -1.000 / 3.0000 / -3.00
          held as AMOUNT_MISMATCH although the amount matches: F0040 / -2.000 / 416.0750 / -832.15
  FAIL  expected_outcomes_v1.0     rows that differ from the expected outcome: 18 of 56
          F0005 (F05): expected ACCEPTED, got QUARANTINED AMOUNT_MISMATCH
          F0006 (F06): expected ACCEPTED, got QUARANTINED AMOUNT_MISMATCH
          F0017 (F17): expected ACCEPTED, got in no output
          F0018 (F18): expected ACCEPTED, got in no output
          F0019 (F19): expected QUARANTINED CONFLICTING_DUPLICATE, got in no output
  PASS  expected_amounts           all 16 compared amounts match
  FAIL  spec_agreement             points where Spark and the plain-Python rules disagree: 20
          F0005: rules say ('ACCEPTED', None), Spark wrote ('QUARANTINED', 'AMOUNT_MISMATCH')
          F0006: rules say ('ACCEPTED', None), Spark wrote ('QUARANTINED', 'AMOUNT_MISMATCH')
          F0017: rules say ('ACCEPTED', None), Spark wrote no row
          F0018: rules say ('ACCEPTED', None), Spark wrote no row
          F0019: rules say ('QUARANTINED', 'CONFLICTING_DUPLICATE'), Spark wrote no row
  passed 6, failed 5, skipped 1
```

CHECKPOINT

All three new checks fail, on the fixture and on the training data. On the training data they report 232 delivered rows in no output, 46 accepted rows of conflicting keys, and 1,682 valid credit lines held as mismatches. Your numbers are these numbers when your checks follow the docstrings. Ten minutes ago, no check failed on the training data.

IF IT DOES NOT WORK

No assistant, or no time: `python tools/checkpoint.py prepared lab3` writes two tests and three checks over your files, in the style of the assistant. The change contains three problems. Review it with `python tools/checkpoint.py diff` before you run it, and compare what its tests and checks say with what the fixture says. `prepared_ai_outputs/Lab3_prepared_tests_and_checks.md` has the response that belongs to it.

Step 4. Trace two failures

One failure to specific records. `reconciliation` names rows that are in no output. `run_trace.py` follows chosen rows through the rules. `lab_config.py` already names three rows of the fixture in `TRACE_ROWS`. Run `run_trace.py` on Databricks:

```
o2i revision <your revision> | contract 1.0 | Spark <version>
input main.o2i | output main.o2i_<your name>

Trace on dataset 'fixture': 3 rows asked for, 2 found

F0005  CN-3000004 line 1
  delivered  status 'POSTED', date '2026-04-03', customer 'C200001', product 'P-20001'
             quantity -4.000 x unit price 91.7875, net amount -367.15
  facts      missing_required False | status_class 'POSTED' | duplicate_class 'UNIQUE' | date_valid True |
      customer_matches 1 | product_matches 1 | amount_ok False
  outcome    QUARANTINED AMOUNT_MISMATCH

F0049  INV-3000016 line 1
  delivered  status 'POSTED', date '2026-04-02', customer 'C200006', product 'P-20001'
             quantity 2.000 x unit price 96.5080, net amount 193.02
  facts      missing_required False | status_class 'POSTED' | duplicate_class 'UNIQUE' | date_valid True |
      customer_matches 1 | product_matches 1 | amount_ok True
  outcome    ACCEPTED

F0017  NOT IN THE RESULT: no rule decided this row. It was lost before the rules.
```

Look up cases F05 and F17 in the fixture workbook. Rows F0017 and F0049 are the same invoice line, delivered twice with identical content.

RECORD YOUR EVIDENCE

What the contract says about F0017 and F0049 (rule R3):

What the code did with them:

Where in the flow the row was lost: before the rules / in a rule / after the rules

One failure to a decision in the code. Send request 3.4 with the output of your own run in place of the angle brackets.

```
The data-quality check signed_amounts fails on the training data. Its output is:
<paste the line of the check and its evidence lines>
Do not edit any file. Read o2i/transform.py and name the one expression that decides AMOUNT_MISMATCH.
Show with the values of <one RawRowID from the evidence> why that line is held.
Then say what the smallest correction is, and which rows would change their outcome.
```

Check the explanation against row F0005 of the trace yourself: quantity `-4.000`, unit price 91.7875, net amount `-367.15`. The explanation must fit the delivered values. An explanation that you cannot reproduce with one row is a guess.

Step 5. Repair one cause at a time

You have named three causes. Fix them in this order, one at a time, so that every change has its own evidence.

Cause	Where	Evidence before	Run after the fix	Evidence after
1. The tax is rounded half to even	`o2i/rules.py`, `tax_amount`	Two failing code tests	`python tools/run_tests.py`	`28 passed, 0 failed, 1 skipped`
2. Only the latest delivery of a line reaches the rules	`o2i/transform.py`, `add_facts`	`reconciliation`, `conflicts_held`	`run_checks.py`	Both pass. On the fixture: `passed 8, failed 3, skipped 1`
3. The amount rule compares the absolute quantity	`o2i/transform.py`, `add_facts`, rule R7	`signed_amounts`	`run_all.py`	`REGRESSION: ALL GREEN`

For each cause, send request 3.5 with your own words for the cause:

```
Make the smallest change in <o2i/rules.py or o2i/transform.py> that corrects this cause:
<the cause, in your own words>
No refactoring and no renaming. Do not change tests, checks, fixtures, or documents.
Then tell me which code tests and which data-quality checks I should run, and what each of them
should show.
```

Judge every fix before you keep it. A fix can turn a failing line green without correcting anything. These four questions tell a correction from a cover-up:

1. Does the change touch a test, a check, a fixture, or an expected value? Then it changes the judge, not the code.
2. Does it remove the symptom by removing rows, for example with a filter, an exemption, or a default?
3. Does it agree with the sentence of the contract that you would quote for it?
4. After the run: did the evidence that you predicted change, and nothing else?

When all three causes are corrected, set `DATASETS = ["fixture", "training"]` and run `run_all.py` on Databricks. These are its last lines:

```
Data-quality checks on dataset 'training'
  PASS  schema_contract           columns and types match the contract
  PASS  required_fields           no NULL in a required output column
  PASS  reconciliation            raw 49,700 = accepted 45,133 + exceptions 4,567
  SKIP  amount_totals             TODO: amount_totals is not written yet (Lab 3, optional check 4)
  PASS  unique_grain              one accepted row for each InvoiceID and LineNumber
  PASS  conflicts_held            conflicting keys delivered: 48; none of them accepted
```

```
  PASS  referential_integrity      every accepted key has exactly one master row
  PASS  status_rule                only posted lines accepted, no posted line turned away for its status
  PASS  signed_amounts             signs and amounts intact; accepted credit lines: 1,680
  SKIP  expected_outcomes          this dataset has no expected results
  SKIP  expected_amounts           this dataset has no expected amounts
  SKIP  spec_agreement             runs on the fixture only: it collects every row
  passed 8, failed 0, skipped 4

CHECKS: ALL PASSED
REGRESSION: ALL GREEN
```

CHECKPOINT

`REGRESSION: ALL GREEN`. The training run accepts 45,133 lines, and `reconciliation` says that every one of the 49,700 delivered rows is in exactly one output. On the fixture, all 56 expected outcomes match. One test and one check are still skipped: they are the optional ones, and a skipped check proves nothing.

IF IT DOES NOT WORK

No assistant, or you want a second opinion: `prepared_ai_outputs/Lab3_prepared_fix_proposals.md` has three candidate fixes for each cause. Each candidate comes with a plausible argument. One of the three corrects the cause. Choose before you apply anything, and say why the other two are wrong.

Step 6. Checkpoint

RECORD YOUR EVIDENCE

Cause	Failed before the fix	Passes after the fix	Why our fix corrects the cause and does not hide the symptom
1			
2			
3			

Three record keys from our exception summary, with their reason codes:

KEY MESSAGE

A passing code test does not authorize acceptance of the dataset. This morning's code tests were green, and 232 delivered rows were missing. A test, a check, and the fixture each see what the others cannot.

If something does not work

You see	Cause	Do this
A new test passes at once	Its expected value was calculated the way the code calculates	Write the expected value as a literal from your hand calculation
A new check passes on the fixture before any fix	It counts something that the defect does not change, or it leaves rows out before it counts	Read the docstring again: `reconciliation` counts `RawRowID` values, not invoice lines
A check fails with an error, not with `FAIL`	An exception inside the check: a column name, or a type	Read the last line of the error. Fix the check, not the data.
`conflicts_held` reports a row that the fixture expects to be accepted	The check ignores the status: a draft and a posted delivery of one key are not a conflict	The docstring says "among the rows whose status is POSTED"
After fix 2, `signed_amounts` still fails	Correct: cause 3 is not fixed yet	Go on
After all fixes, `expected_outcomes` still names rows	One of the fixes replaced the defect by another rule	Compare the named cases with the fixture workbook, and with the contract
`python tools/run_tests.py` says `skipped` and you wrote the test	The line `self.skipTest(...)` is still in the test	Remove the line

> **IF YOU ARE AHEAD**
>
> Write the optional test and the optional check: `test_amounts_equal_the_hand_calculation` in `tests/unit/test_fixture.py`, and `amount_totals` in `o2i/checks.py`. With both, the suite says `29 passed, 0 failed` and no check is skipped. Then open the fix proposals of the prepared outputs and name, for each cause, the wrong candidate that would be hardest to catch in a review.
>
> **A design question.** Rule R3 holds every delivery of a conflicting line. What would "the latest delivery wins" cost the business, and who may decide it?

Lab 4: Extend without regression

Module 5 · 28 minutes · in pairs · your choice of tool · then 12 minutes of peer review

The business changes a rule. From now on, an invoice line must point to exactly one order line, or it is reported. You take the change from the request to a result that a reviewer can accept: acceptance cases first, a failing test, a bounded change, a full regression run, and an explanation in your own words.

YOU PRACTICE	YOU START FROM	YOU FINISH WITH
The whole loop on a change that nobody prepared for you	The checkpoint `lab4-start`: the verified result of Lab 3, `29 passed, 0 failed`	Rule R8, `REGRESSION: ALL GREEN` on three datasets, and a review note

The change

The request is in your project: `docs/change_request_CR-2026-041.md`. Read it first. It is one page, and every sentence of it decides a case.

R8 Order reference, checked after rule R7. A line passes when `OrderID` and `OrderLineNumber` are present and match exactly one row of the `orders` table.

Situation	Disposition	Reason code
`OrderID` is NULL or blank, or `OrderLineNumber` is NULL	`QUARANTINED`	`MISSING_ORDER_REF`
No row of `orders` has this `OrderID` and `OrderLineNumber`	`QUARANTINED`	`UNKNOWN_ORDER_REF`
More than one row of `orders` has this `OrderID` and `OrderLineNumber`	`QUARANTINED`	`AMBIGUOUS_ORDER_REF`

The rule checks that the order line exists. It does not compare quantity, price, customer, or product. A valid credit keeps its signed amounts and is accepted when every rule passes, including R8.

Two datasets judge the change: the fixture, where cases F36 to F42 belong to this rule, and the challenge batch, 191 unseen rows in 16 cases with expected outcomes.

Minutes	Step	You do	Decided by
0 to 3	Get ready	Load the checkpoint, read the change request	`29 passed, 0 failed`
3 to 7	1. Acceptance cases	List the cases before any code exists. No edit.	The change request
7 to 12	2. Make it fail	Set the contract to 1.1, write the tests, run the checks	`23 passed, 6 failed`, and two failing checks
12 to 20	3. Change	Choose where to work. One bounded change. Review the difference.	Your four review answers
20 to 25	4. Validate	Code tests, then the full suite on three datasets	`REGRESSION: ALL GREEN`
25 to 28	5. Review note	Write the handoff for the peer review	Numbers from your runs

Get ready

1. Load the checkpoint, and run the code tests.

```
python tools/checkpoint.py lab4-start
python tools/run_tests.py
```

> **CHECKPOINT**
> `29 passed, 0 failed`. This is the verified state of Lab 3 with the optional test and check written, so nothing is skipped. Everybody starts Lab 4 from the same code, whatever happened in Lab 3.

2. Read `docs/change_request_CR-2026-041.md`, two minutes, both of you. Mark the section "What the rule does not do".

Step 1. Acceptance cases, before any code

Send request 4.1 to the assistant of your choice. It edits nothing.

```
Read docs/change_request_CR-2026-041.md and docs/business_contract.md. Do not edit any file.
List the acceptance cases of rule R8 as a table: situation, example values, expected disposition,
expected reason code. Include a case in which an earlier rule decides first, a valid credit, and an
exact replay. Then list what the change request says the rule does not do.
```

Compare the table with the fixture workbook, cases F36 to F42. Then answer three questions yourselves, before the code can answer them for you:

> **RECORD YOUR EVIDENCE**
>
> A line has a wrong amount and an unknown order reference. Its reason code: _____
>
> A credit points to an order line that does not exist. Its outcome: _____
>
> The order line exists, and it belongs to another customer. Its outcome: _____

Step 2. Make it fail first

A change that was never red has no evidence that its tests can fail.

1. **Tell the code which contract applies.** Open `o2i/config.py` and make these two edits by hand:

```
CONTRACT_VERSION = "1.1"
```

   and add the three new reason codes at the end of the tuple `REASONS`:

```
    "MISSING_ORDER_REF", "UNKNOWN_ORDER_REF", "AMBIGUOUS_ORDER_REF"
```

   From now on, the fixture test and the check `expected_outcomes` compare with the expected results of version 1.1.

2. **Write the tests of the new rule.** Send request 4.2.

```
In tests/unit/test_rules.py, add the decision tests for rule R8. Edit only this file.
- In the helper decide(), the clean line gets order_ref_present=True and order_matches=1.
- test_each_rule_has_its_reason: add MISSING_ORDER_REF (order_ref_present False), UNKNOWN_ORDER_REF
  (order_matches 0), and AMBIGUOUS_ORDER_REF (order_matches 2). All three are QUARANTINED.
- test_the_first_failing_rule_decides: add one case that shows rule R7 before rule R8.
- test_every_reason_is_a_reason_of_the_contract: add the three new situations.
These tests will fail until the rule exists. That is intended. Do not change o2i/. Do not run anything.
```

Review: the three new reason codes are literals, the case for R7 before R8 expects `AMOUNT_MISMATCH`, and no existing assertion changed. Then run `python tools/run_tests.py`.

> **CHECKPOINT**
> `23 passed, 6 failed`. The decision tests fail because `decide_disposition` does not know the two new facts yet. The fixture test fails for cases F36, F37, F38, F39, and F41: the code still accepts them.

3. **See the failure in the data.** In `lab_config.py`, set `DATASETS = ["fixture", "challenge"]`. Run `run_checks.py` on Databricks.

```
  FAIL  expected_outcomes_v1.1     rows that differ from the expected outcome: 22 of 191
          X0009 (C09): expected QUARANTINED UNKNOWN_ORDER_REF, got ACCEPTED
          X0010 (C09): expected QUARANTINED UNKNOWN_ORDER_REF, got ACCEPTED
          X0027 (C04): expected QUARANTINED UNKNOWN_ORDER_REF, got ACCEPTED
          X0030 (C07): expected QUARANTINED AMBIGUOUS_ORDER_REF, got ACCEPTED
          X0032 (C07): expected QUARANTINED AMBIGUOUS_ORDER_REF, got ACCEPTED
```

> **CHECKPOINT**
> The check `expected_outcomes_v1.1` fails on both datasets: 5 of 56 fixture rows and 22 of 191 rows of the challenge batch are accepted and should be held. Every other check still passes. That is the before picture of your review note.

Step 3. Choose where to work, then make one bounded change

Switch driver and reviewer now.

Choose first. Both assistants can do this change. They start from different truths.

Part of the work	A good starting point	Because
Understand `orders`: which keys repeat, which invoice lines have no reference	Azure Databricks with Genie Code	It sees the table, the column comments, and the result of a query
Change two modules and their tests together	VS Code with GitHub Copilot	It sees the whole project, the tests, and `AGENTS.md`
Prove the result	Databricks compute	The data is there

> **RECORD YOUR EVIDENCE**
>
> We work in: _____
>
> Because: _____

Then send request 4.3. In VS Code, use the agent. In the workspace, open the project file in the editor and use Genie Code: it reads `AGENTS.md` by itself.

```
Implement rule R8 of docs/change_request_CR-2026-041.md. Read AGENTS.md and the change request first.
Scope:
- o2i/rules.py: decide_disposition gets two keyword arguments, order_ref_present and order_matches,
  and three outcomes after rule R7. evaluate_rows gets the order lines as a fourth argument and
  computes both facts for every row.
- o2i/transform.py: read the orders table. In add_facts, add the facts f_order_ref_present and
  f_order_matches: count the order lines for each OrderID and OrderLineNumber and join the count, as
  the customer and product rules do. Extend decide after rule R7. No inner join and no filter.
- Update every caller of a function whose signature you change, in o2i/ and in tests/.
Do not change the order or the meaning of rules R1 to R7. Do not change an expected value, a fixture
file, or an assertion that exists today.
The rule checks that the order line exists, with OrderID and OrderLineNumber together. It does not
compare quantity, price, customer, product, or order status.
Do not run anything. When you are done, list every file that you changed. For each new line of the
decision, name the acceptance case that it serves.
```

Review the difference before you keep it. This change is larger than any before, so read it with `python tools/checkpoint.py diff`, or in the Source Control view, file by file.

1. **Scope.** Which files changed? In `tests/`, the only allowed change is the new argument of a call. No expected value, no skipped case, no new import, and no new dependency.
2. **Order.** Is R8 decided after R7, in `o2i/rules.py` and in `o2i/transform.py`? What does a line with a wrong amount and an unknown order get?
3. **The join.** Are the order lines counted for each key before the join? A join to the `orders` table itself can multiply an invoice line, and a `dropDuplicates` on the orders makes `AMBIGUOUS_ORDER_REF` impossible.
4. **Exceptions and defaults.** Is any group of lines exempt, for example credits? Do the two new arguments have default values? A default that means "passes" lets a caller forget the rule without an error.

> **IF IT DOES NOT WORK**
>
> No assistant: `python tools/checkpoint.py prepared lab4` writes a prepared change over the checkpoint, in the style of the assistant. Its summary says that all code tests pass, and they do. The change contains six problems. Review it with the four questions, run step 4, and repair what the fixture and the challenge batch report. `prepared_ai_outputs/Lab4_prepared_change.md` has the response that belongs to it.

Step 4. Validate

1. Run the code tests on your workstation.

   > **CHECKPOINT**
   >
   > `29 passed, 0 failed`. The tests that failed in step 2 pass, and every earlier test still passes.

2. In `lab_config.py`, set `DATASETS = ["fixture", "challenge", "training"]`. Run `run_all.py` on Databricks. This is the block of the challenge batch, and the end of the run:

   ```
   Dataset 'challenge': 191 rows written to main.o2i_<your name>.challenge_accepted and main.o2i_<your
       name>.challenge_exceptions
     ACCEPTED                                156
     EXCLUDED     EXACT_REPLAY                 3
     EXCLUDED     NOT_POSTED                   2
   ```

```
  QUARANTINED  AMBIGUOUS_ORDER_REF             3
  QUARANTINED  AMOUNT_MISMATCH                 1
  QUARANTINED  CONFLICTING_DUPLICATE           4
  QUARANTINED  INVALID_DATE                    1
  QUARANTINED  MISSING_ORDER_REF               6
  QUARANTINED  UNKNOWN_CUSTOMER                2
  QUARANTINED  UNKNOWN_ORDER_REF              13
Data-quality checks on dataset 'challenge'
  PASS  schema_contract            columns and types match the contract
  PASS  required_fields            no NULL in a required output column
  PASS  reconciliation             raw 191 = accepted 156 + exceptions 35
  PASS  amount_totals              delivered totals equal written totals in every currency
  PASS  unique_grain               one accepted row for each InvoiceID and LineNumber
  PASS  conflicts_held             conflicting keys delivered: 2; none of them accepted
  PASS  referential_integrity      every accepted key has exactly one master row
  PASS  order_reference            every accepted line references exactly one order line
  PASS  status_rule                only posted lines accepted, no posted line turned away for its status
  PASS  signed_amounts             signs and amounts intact; accepted credit lines: 8
  PASS  expected_outcomes_v1.1     all 191 expected outcomes match
  SKIP  expected_amounts           this dataset has no expected amounts
  SKIP  spec_agreement             runs on the fixture only: it collects every row
  passed 11, failed 0, skipped 2
...
CHECKS: ALL PASSED
REGRESSION: ALL GREEN
```

Your block has one line less when you did not write the optional check `order_reference`.

3. Compare the fingerprint of the run with `python tools/revision.py`.

> **CHECKPOINT**
> `REGRESSION: ALL GREEN`, and both fingerprints are equal. On the training data, rule R8 now holds 107 lines without an order reference, 116 with an unknown one, and 6 with an ambiguous one. 44,904 lines are accepted, and `reconciliation` still accounts for every one of the 49,700 delivered rows.

To see one row in detail, name it in `TRACE_ROWS` of `lab_config.py` and run `run_trace.py`. For the fixture row F0042, the wrong amount decides before the order reference:

```
F0042  INV-3000041 line 1
  delivered  status 'POSTED', date '2026-04-02', customer 'C200001', product 'P-20001'
             quantity 4.000 x unit price 91.7875, net amount 376.15
  facts      missing_required False | status_class 'POSTED' | duplicate_class 'UNIQUE' | date_valid True |
      customer_matches 1 | product_matches 1 | amount_ok False | order_ref_present True | order_matches 0
  outcome    QUARANTINED AMOUNT_MISMATCH
```

Step 5. Write the review note

Open `takeaways/4_review_note_template.md`, copy the block into a new file `review_note.md` in the project folder, and fill it in. Three minutes. Every number in it comes from a run that you can repeat. "Remaining risks and limits" is never empty: what does rule R8 not check?

> **IF YOU ARE AHEAD**
>
> Request 4.4 of `prompts/LAB_PROMPTS.md` adds a data-quality check `order_reference` for the new rule. Request 4.5 updates the contract document to version 1.1. Then trace row `X0023` of the challenge case C16: set `TRACE_DATASET = "challenge"` and `TRACE_ROWS = ["X0023"]` in `lab_config.py`, and run `run_trace.py`. Explain its outcome to your partner: the order line exists, and it belongs to another customer.
>
> **A design question.** Where does a comparison of the customer of the order line belong: in rule R8, in a new rule, or in a report?

If something does not work

You see	Cause	Do this
`TypeError` in many tests after the change	A caller was not updated, or an argument has another name than in the tests	The tests are the specification: `order_ref_present` and `order_matches`
Fixture case F42 fails after the change	R8 is decided before R7	Move the three new outcomes after the amount rule, in both files
Fixture case F39 or challenge case C07 fails	The order lines were made unique before the count	Count the rows for each key. Do not remove any.
Fixture case F41 or challenge case C09 fails	Credits are exempt from the rule	The change request says that a credit is accepted "when every rule passes, including R8"
`reconciliation` fails with rows written more than once	The transformation joins to `orders`, not to a count for each key	Aggregate first, then join, as rules R5 and R6 do
The code tests pass and `expected_outcomes_v1.1` fails	Spark and plain Python disagree, or a test skips cases	Read `spec_agreement`, and read the difference of `tests/` again
The run still says `contract 1.0`	`o2i/config.py` was not saved, or the run used an older copy	Save all files, and compare the fingerprints

Peer review: explain it without the assistant

Module 5 · 12 minutes · two pairs together · two rounds of six minutes

A reviewer accepts a change from a person, not from a tool. For two minutes you explain your change and its evidence with the chat closed. Then two questions test whether the explanation holds. Then you switch roles.

Minutes	Who	Does
2	The developers	One explains the change, the other the evidence, from the review note. No assistant.
2	The reviewers	One asks about the business rule, the other about the tests and the evidence
2	Both pairs	Close one gap: a missing piece of evidence, a wrong sentence, an untested case

A question about the business rule	A question about the tests and the evidence
What happens to a credit whose order line is unknown?	Which test failed before your change, and why?
Wrong amount and no order reference: which code, and why?	Show me the row that proves `AMBIGUOUS_ORDER_REF`.
The order exists, and the order line does not. What happens?	What proves that the join lost or multiplied no line?
An order line of another customer: does the line pass? Should it?	What did you not test?

The performance check: five conditions, all required

The reviewers fill this in for the other pair.

Condition	It is met when	Shown
Correctness	The order-reference rule matches the expected cases of the fixture and of the challenge batch	☐
Regression protection	A test or check showed the new failure. It passes now, and every earlier test and check still passes.	☐
Data accountability	No silent loss and no fan-out. Accepted rows and exception rows reconcile with the delivered rows.	☐
Cross-tool fidelity	The revision that was reviewed is the revision that was validated on Databricks	☐
Human understanding	The developers explained behavior and limits without reading an AI answer	☐

> Verdict: review ready / needs rework
>
> The gap that we closed together:

Fewer than five means "needs rework", not "review ready". So does a failing check, behavior that nobody can explain, or a dependency that nobody approved. This is a learning check. It is not a release approval, and it replaces nothing in the review process of your team.

Apply it tomorrow

Module 6 · 15 minutes · three parts: write for two minutes, then one minute for each partner

Choose the tool for your next real task

My task, one that is on my desk this week:

I start in: Databricks with Genie Code / VS Code with GitHub Copilot / a deliberate combination

Question	My answer for this task
Context: what must the assistant see?	
Execution: what must run to prove the result?	
Maintainability: who has to live with the result?	
Cost: what is metered, and what would rework cost?	

Prepare the handoff

Your team's review process stays as it is. Write the three lines of a review note that are hardest to write.

The evidence that my reviewer will ask for:

A remaining risk that I will name myself:

A question for my tech lead that is not mine to decide:

Commit to your next task

My commitment	
One task to try	
A reusable prompt, from takeaway 1	
The required quality check	
The intended human reviewer, by name	
By when	

KEY MESSAGE

An assistant may accelerate the work. The developer must understand and validate the result. Measure the time to a verified result, and the rework.

Takeaway 1: Prompt patterns

A prompt is a specification. An assistant fills every gap in it with an assumption, and it does not tell you which ones unless you ask.

The bounded request: six ingredients

Ingredient	Write down	In the workshop case
Business intent	What the result is for, and who uses it	Accepted invoice lines for a revenue report
Input schema and grain	The tables, what one row is, the keys, and what nothing guarantees	One row is one delivery of an invoice line. A line can be delivered more than once. No master key is enforced.
Expected result	Columns, types, and grain of the output, and where every other row goes	One row for each accepted line. Every other delivered row in an exception output, with a reason code.
Permitted libraries	Language, API, and what must not be added	PySpark DataFrame API or SQL. No pandas, no UDF, no new package.
Error behavior	What happens to NULL, an unknown key, a duplicate key, an invalid value	Never drop a row, never fill a value, never choose one of several rows.
Acceptance criteria	The checks that you will run, with numbers where you have them	Delivered rows = accepted rows + exception rows. Totals for each currency equal the validated totals.

Two habits belong to every request:

- **Assumptions first.** Ask for the assumptions before the implementation, and answer them.
- **Small steps.** Ask for one change that you can review in two minutes, not for a solution.

The template

```
Goal:        <what the result is for, in one sentence>
Input:       <tables or DataFrames, what one row is, the keys, what is not guaranteed>
Rules:       <the business rules that apply, in the order in which they decide>
Result:      <columns, types, and grain of the output. Where do the other rows go?>
Constraints: <language and API, permitted libraries, what must not change>
Errors:      <NULL, unknown key, duplicate key, invalid value: what must happen>
Acceptance:  <the checks that I will run, with numbers where I have them>
Working:     Change only <scope>. Do not run anything.
             First list your assumptions and your questions, and wait for my answer.
```

Nine patterns

Replace the text in angle brackets. Each pattern ends with what you do before you accept the answer.

1. Assumptions first

```
Do not write code yet. List the assumptions that you would have to make about <the data or the rule>,
and the questions that you have for me. Mark each assumption as: stated in my request, visible in
the schema, or a guess.
```

Before you accept: answer every guess from the schema, the contract, or the data. A guess that you cannot answer is a question for the owner of the data, not for the assistant.

2. One small step

```
Change only <function, cell, or file>. Keep its signature. Do not touch <tests, fixtures, documents,
other files>. No new import and no new dependency. Do not run anything.
After the change, list what you changed and why.
```

Before you accept: read the diff. A file that you did not name must not be in it.

3. Explain, do not run

```
Just explain this to me, do not run any code and do not change anything.
What does <expression or function> return for: NULL, a blank text, a negative amount, and a key that
matches two rows? Answer as a table: input, result, why.
```

Before you accept: pick one row of the table and prove it, with a test or with one query.

4. Move without changing the meaning

```
Move the logic of <source> into <target>. The meaning must not change: the same rows, the same
values, the same types. Do not improve, rename, or simplify anything on the way.
List every place where you had to make a choice.
```

Before you accept: compare the old result and the new result in five ways: schemas, key sets, row counts, signed amounts, and aggregates. A matching grand total proves nothing.

5. Test cases from the contract, not from the code

```
Read <the contract> and <the existing tests>. Do not take any expected value from the implementation.
List the cases that no test covers yet. For each case: the rule, concrete input values, the expected
result, and how you derived it from the contract.
Mark every expected result that you calculated yourself with CHECK BY HAND.
```

Before you accept: calculate every expected value yourself. An expected value that comes from the code under test preserves the defect that the test should find.

6. Tests with explicit expected values

```
Write tests for these cases, which I verified by hand: <cases with expected values>.
Use literal expected values. Do not compute an expected value with the function under test.
Do not change the implementation. Do not run anything.
```

Before you accept: run the tests against the unchanged code. A new test that passes at once has either found nothing or tests nothing. Find out which.

7. Trace a failure to a decision in the code

```
The check <name> fails. Its output is: <paste the output, with the record keys>.
Do not edit any file. Name the one expression in <file> that produces this outcome. Show with the
values of <record key> why it does. Say which rows would change their outcome if it were corrected.
```

Before you accept: look at the record yourself. The explanation must fit the delivered values.

8. The smallest fix

```
Make the smallest change that corrects the cause that you named. No refactoring and no renaming.
Do not change tests, checks, fixtures, or documents.
Then tell me which tests and which checks I should run, and what each of them should show.
```

Before you accept: a fix that edits a test, weakens a check, or filters out the rows that failed hides the symptom. Decline it.

9. Translate and compare

```
Translate only <expression> from <SQL> to <PySpark>. Do not change its meaning.
Tell me what your version returns for NULL, for a blank text, and for <an edge value>.
```

Before you accept: run both versions on the same rows and compare them row by row. Two versions that read alike are not two versions that agree.

Takeaway 2: Code and data review checklist

Use it for a change that an assistant wrote, and for a change that you wrote. The questions are the same.

Before it runs: inspect

Look for	Because
A file, function, or cell that you did not name	The change is larger than the request
A new import, package, UDF, shell command, or network call	Dependencies and commands need approval, not momentum
`filter`, an inner join, `dropDuplicates`, `dropna`, `limit`	Each of them can lose rows without an error
A join to a table whose key is not proven unique	One line becomes two, and every total grows
A delivered value that is overwritten, trimmed, filled, or defaulted	The evidence is gone, and the report looks cleaner than the data
`double` or `float` for money, `abs()`, a filter on positive amounts, arithmetic with NULL	Precision, signs, and missing values change totals silently
An edited test, expected value, fixture, or contract	The oracle was changed to fit the code

After it ran: validate

"The code ran" is not evidence. These six questions are the test matrix of the workshop.

Question	Evidence that answers it
Behavior and precision: does each function do what the rule says?	Code tests with expected values calculated by hand. Include a NULL, a rounding tie, and a credit.
Schema and required fields: does the output have the agreed shape?	Columns and types equal the contract. No NULL in a column that must be filled.
Uniqueness and replay: is every business key in the output once?	One accepted row for each key. Replays are documented. Conflicting rows are held, not chosen.
Relationships and joins: did a join lose or multiply rows?	Every accepted key has exactly one master row. Row counts by key before and after each join.
Business rules: are the right rows in and the right rows out?	Only allowed status values are accepted. A valid credit is accepted with its sign.
Reconciliation and regression: is every row accounted for, and did nothing else move?	Delivered rows = accepted + exceptions, by row key. Totals for each currency. Fixture, parity, and the full suite pass.

Before you say "review ready": explain

All five, or the verdict is "needs rework".

1. **Correctness.** The result matches expected cases that did not come from the code.
2. **Regression protection.** A test or check failed before the change and passes after it. Everything that passed before still passes.
3. **Data accountability.** No silent loss and no fan-out. Accepted rows and exception rows reconcile with the delivered rows.
4. **Cross-tool fidelity.** The revision that you reviewed is the revision that ran on Databricks.
5. **Human understanding.** You can explain the behavior and its limits without reading an AI answer.

Three things you never do to get a green run: delete a failing assertion, skip it, or weaken it.

Takeaway 3: Tool-selection guide

Choose the tool that has the right truth in its context. Then check where the code actually runs.

Where to start

Task	Start with	Evidence, and the boundary
Understand a table, or debug the result of a notebook	Databricks with Genie Code	It sees the schema, the comments, and the output of a cell. Review the actual schema and the actual output.
Extract a helper, restructure a module, write a test	VS Code with GitHub Copilot	It sees the whole project and its instructions. Inspect a small diff and run the local code tests.
Validate joins, volumes, and governed data access	Approved Databricks compute	Run the transformation and the data-quality checks on the real tables.
Accept a change for technical review	A developer and a reviewer	Explain the behavior, the tests, the risks, and the limits. No tool does this for you.

Four questions for your next task

Question	It points to Databricks when	It points to VS Code when
Context: what must the assistant see?	The answer is in the data, a table comment, or a cell output	The answer is in several files, tests, or project instructions
Execution: what must run to prove the result?	Spark, tables, governed data	Plain Python functions and their tests
Maintainability: who has to live with the result?	A one-off analysis or an exploration	Code that will be reviewed, versioned, and reused
Cost: what is metered?	Genie Code usage for each user, and the compute that runs the cells	Copilot usage from the organization's pool. Spark work still runs on Databricks compute.

Neither tool is always cheaper. A vague request that needs three corrections costs more than a bounded request, in both tools. Measure time to a verified result, and rework. Do not measure generated lines.

What runs where

You do this	It runs
Edit a file in VS Code	On your workstation
Run the code tests (`python tools/run_tests.py`)	On your workstation. Plain Python, no Spark.
Run a `run_*.py` file on Databricks from VS Code	On Databricks compute. Your workstation sends the files and shows the output.
Use Databricks Connect	Your Python code runs on the workstation. Every Spark operation runs on Databricks compute.
Run a notebook cell	On Databricks compute

Editing locally does not make Spark local, and it does not make it free.

One source of truth

- A notebook prototype stays in a scratch location.
- After the extraction, the version-controlled project owns the code.
- The folder that the IDE synchronizes to the workspace is a one-way mirror. Never edit the mirror.
- The revision ties both sides together: the revision that you reviewed must be the revision that ran.

Takeaway 4: Review note

A review note is the handoff into the review process that your team already has. It replaces nothing: the pull request, the reviewer, and the approval stay as they are. Keep it to one screen.

```
REVIEW NOTE

Purpose
  <What the change is for, in one or two sentences. Name the requirement or the change request.>

Code revision
  <Commit or revision that was reviewed>     <The same revision, as printed by the run on Databricks>

Changed behavior
  <What is different for the data now. Name rules, reason codes, columns.>
  <What did not change, where a reader could expect a change.>

Test and data evidence
  Before:  <the test or check that failed, with the number or the record keys>
  After:   <the same test or check, passing>
  Regression: <code tests: n passed. Data-quality checks: n passed, on which datasets.>
  Accountability: <delivered rows = accepted + exceptions, with the three numbers>

Remaining risks and limits
  <What the change does not do. What was not tested. What depends on the environment.>

AI assistance
  <Which assistant wrote or proposed what. What you declined or corrected, and why.>

Questions for the tech lead
  <Decisions that are not yours: a business rule, a data owner, a release step.>

Reviewer: <name>          Date: <date>
```

Three rules for the note:

1. Every number in it comes from a run that you can repeat, not from an AI answer.
2. "Remaining risks" is never empty. A change without limits has not been examined.
3. If you cannot fill in "Changed behavior" without the assistant open, the change is not ready.

The four takeaways are also files in your project, in the folder `takeaways`. Copy them to where your team keeps its working agreements.

Reference

The project at a glance

You want to	Use
Read the rules	`docs/business_contract.md`
Look up a column	`docs/data_dictionary.md`
Look up a fixture case and its hand calculation	`02I_Fixture_Workbook.xlsx`
Paste a request of a lab	`prompts/LAB_PROMPTS.md`
Run the code tests on your workstation	`python tools/run_tests.py`
See one line for each test	`python tools/run_tests.py -v`
See which revision you have	`python tools/revision.py`
See what changed since the checkpoint	`python tools/checkpoint.py diff`
List the checkpoints, and see which one is loaded	`python tools/checkpoint.py list`
Load the starting point of a lab	`python tools/checkpoint.py lab3-start`
Work without an assistant	`python tools/checkpoint.py prepared lab3`, and the folder `prepared_ai_outputs`
Run the transformation on Databricks	`run_transform.py`
Run the transformation and every data-quality check	`run_checks.py`
Run the full regression suite	`run_all.py`
Follow chosen rows through the rules	`run_trace.py`, with `TRACE_DATASET` and `TRACE_ROWS` in `lab_config.py`
Choose the datasets of a run	`DATASETS` in `lab_config.py`: `"fixture"`, `"training"`, `"challenge"`

A checkpoint replaces the code and the documents of the project: `o2i/`, `tests/`, `scratch/`, `docs/`, and the `run_*.py` files. It never touches `lab_config.py`, `databricks.yml`, or your settings. Before it replaces anything, it copies your files to `.backups`.

Genie Code in one table

You want to	Do this
Open the pane	The sparkle icon in the upper-right corner of the page
Start without the history of the last task	**New chat** in the header of the pane
Point to a table, a file, or a cell	Type `@` and choose it
Ask inside a cell	`Ctrl+I` in the cell
Get an explanation and nothing else	End the prompt with: "Just explain this to me, don't run any code."

You want to	Do this
Use a shortcut for a common prompt	Type `/` : `/explain` , `/fix` , `/doc` , `/optimize` , `/prettify` , `/findTables` , `/findQueries`
Decide on a request for permission	**Allow** or **Skip**, after you read what it wants to do
Decide at a stop of a longer task	**Continue**, or **Reject** to make it try something else
Keep or refuse a proposed change of `/fix` or `/doc`	**Accept** or **Reject** in the difference view. Accepted code does not run by itself.
Check how it asks	**More** > **Settings** > **Actions**: **Ask first**

Genie Code plans, runs code, reads the output of cells, and tries again after an error. That is why the approval request exists, and why you read it. Start a new chat for each task: a long chat carries its whole history into every request.

GitHub Copilot in VS Code in one table

You want to	Do this
Open the Chat view	`Ctrl+Alt+I`, or the **Chat** menu in the title bar > **Open Chat**
Start a new chat	**New Chat (+)**
Add a file as context	Type `#` and the name of the file, select **Add Context**, or drag the file into the chat
Ask without an edit	**Ask** in the **Agent** control where it is offered, and "Do not edit any file" in the request
Ask about a few lines, in place	Select the lines, then `Ctrl+I` for inline chat
Use a shortcut for a common prompt	Type `/` : `/explain` , `/fix` , `/tests`
Accept or refuse an inline suggestion while you type	`Tab` accepts, `Esc` dismisses
Decide on a command that the assistant wants to run	Read it, then **Allow** for this one command
Review an edit	Local session: **Keep** or **Undo**. Copilot session: open the changed file to see the difference. Always: `python tools/checkpoint.py diff`
Take back everything since a request	Hover over the request in the chat and select **Restore Checkpoint**. It restores files, not commands that already ran.
Stop a running request	**Stop** in the chat input
See which instructions were used	Expand **References** in the response

The assistant reads `AGENTS.md` at the root of the project. The file holds the layout of the project, nine hard rules, and the way of working. Instructions guide an assistant. They do not replace your review of the difference.

Words of this course

Word	Meaning
Accepted	A line that passed every rule. It is in the accepted output.
Quarantined	A line that cannot be trusted. A person must decide. It is in the exception output with a reason code.
Excluded	A line that is correctly left out, with a documented reason: not posted, or an exact replay
Grain	What one row of a table is. The grain of the accepted output is one invoice line: `InvoiceID` + `LineNumber`.
Exact replay	A line that was delivered again with identical business content. The first delivery counts.
Conflicting duplicate	The same line delivered with different content. Nobody chooses: all deliveries are held.
Fan-out	A join that writes one input row more than once, because the other side has several rows for the key
Fixture	A small dataset that you can verify by hand, with an expected outcome for every row
Oracle	The source of an expected value. In this course: the rule sheet and the fixture, never the code under test.
Code test	A test of a function, in plain Python, on your workstation
Data-quality check	A check of written tables against the contract, on Databricks
Parity	Two results agree in schemas, key sets, row counts, signed amounts, and aggregates
Reconciliation	Delivered rows = accepted rows + exception rows, counted by `RawRowID`
Revision, fingerprint	Twelve characters that identify the code of the `o2i` package. Equal fingerprints: the code that you reviewed is the code that ran.
Checkpoint	A verified state of the project code that a tool can load
Session target	The control of the Copilot chat that decides which engine runs the session, and how its edits are reviewed

Run route D: the project in the workspace

Use this route when VS Code on your workstation cannot run a file on Databricks. Everything happens in the browser. The notebook `Run_project_in_workspace` runs the same files that the other routes run, from a copy of the project in your workspace folder.

Once: unpack the project

1. Open the notebook `Run_project_in_workspace` and attach it to the compute of your class.
2. Run the first code cell. It creates five widgets at the top of the notebook.
3. Set widget 1, **Action**, to `unpack project`, and select **Run all**.

```
Unpacked 45 files to /Workspace/Users/<your sign-in name>/o2i-workshop
Next: set widget 1 to 'setup check' and select Run all.
```

The project is now a folder `o2i-workshop` in your home folder of the workspace. On this route, that folder is the source of truth: you edit its files in the workspace file editor, with Genie Code as the assistant. Genie Code reads the `AGENTS.md` of the folder by itself.

Every time: choose an action, then Run all

In the labs	Widget 1, Action	Widget 2
`python tools/setup_check.py`	`setup check`	
`python tools/run_tests.py`	`code tests`	
`python tools/revision.py`	`revision`	
Run `run_transform.py` on Databricks, and the same for the other `run_*.py` files	`run_transform`, `run_parity`, `run_checks`, `run_all`, `run_trace`	
Run `demo/parity_trap.py` on Databricks	`parity_trap`	
`python tools/checkpoint.py list`	`checkpoint list`	
`python tools/checkpoint.py diff`	`checkpoint diff`	
`python tools/checkpoint.py lab3-start`	`checkpoint load`	`lab3-start`
`python tools/checkpoint.py prepared lab3`	`checkpoint prepared`	`lab3`

The output appears under the last cell. A setup check on this route ends like this:

```
OK    Python <your version> (3.9 or later is needed)
OK    Code tests ran: 4 passed, 8 failed
OK    Workspace in databricks.yml: <the workspace URL of your class>
OK    Catalog in lab_config.py: main
NOTE  This project runs in the workspace. No Git repository is created here: review changes with the action
    'checkpoint diff'.
OK    o2i revision 1cc70d2ac45c
READY
```

What is different on this route

Topic	On route D
Where you edit	In the workspace: **Workspace** > your home folder > `o2i-workshop`. A file opens in the editor when you select it.
The assistant	Genie Code, in the pane of the file editor. The requests of `prompts/LAB_PROMPTS.md` work as they are.
Reviewing a change	The action `checkpoint diff`. There is no Git repository in the workspace folder.
The code tests	Run on Databricks compute, not on your workstation. Say so in your review note.
The fingerprint	The action `revision` and the line `o2i revision` of a run read the same folder. They are equal by construction, so cross-tool fidelity means: the files in this folder are the ones that you reviewed.
Unpacking again	Overwrites your changes. The notebook refuses unless you type `REPLACE` in widget 5.

If the Databricks extension in VS Code synchronizes your files and only the run fails, you do not need to unpack: enter the path of the row **Remote Folder** of the extension in widget 3. Then you edit in VS Code, and the notebook runs the synchronized copy.